CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING (C-DAC)

Internship Project Report

Development of an Web Crawler for Automated Knowledge Base Creation and RAG-Based Information Retrieval

Page-level provenance, hybrid BM25 and vector retrieval, and evidence-grounded answers over PDF documents

Ingestion of PDF bytes, page-wise extraction, optional OCR, section-aware chunking, 384-dimensional MiniLM embeddings, Elasticsearch dense vectors, min-max hybrid ranking, and a grounded language-model answer with a machine-readable source list.

Internship period: 15 June 2026 to 15 September 2026 (3 months)

Table 1. Cover facts taken from the repository, the live index, and the saved demo

Item	Value
Submitted by	Vyom Desai
Programme / Centre	Internship, Pune
Internship guide	Pavan Dhote, Professor
Organisation	Centre for Development of Advanced Computing (C-DAC)
Department context	Internship project, advanced computing
Repository	C:\CDAC\RAG
Application name	C-DAC Unified RAG
Document type	Internship project report
Document version	1.0
Internship period	15 June 2026 to 15 September 2026 (3 months)
Implementation status	PDF path implemented; live index verified; labelled retrieval set still pending
Active embedding	sentence-transformers/all-MiniLM-L6-v2, 384 dimensions, provider local
Active retrieval	hybrid, keyword 0.3, semantic 0.7, fusion min_max, top_k 5, num_candidates 50
Active index	Elasticsearch 8.12.2, index rag_corpus_v1, 12,054 documents
Recorded PDF corpus	60 PDFs, 1,479 pages, 2,880 text chunks, 2,880 summaries, 316 composites
Recorded answers	docs/demo-results: five hybrid questions, Llama 3.2 through Ollama, none abstained
Test re-run	281 passed, 2 skipped, 7 teardown errors, 74.85 seconds

Status labels used throughout this report are DONE, PARTIAL, PENDING, and NEEDS LIVE VALIDATION. A claim is tied to a module, a configuration key, a test, an artifact, or a saved demo file.

Declaration

I, Vyom Desai, declare that this report describes the PDF retrieval-augmented generation path implemented in the repository C:\CDAC\RAG. The description was prepared from the source code, configuration, tests, extracted artifacts, the Elasticsearch index, and the answers stored in docs/demo-results. Where a behaviour is implemented and switched off, or implemented and not yet exercised against a live dependency, the report says so.

Place: Pune

Internship period: 15 June 2026 to 15 September 2026

Vyom Desai

Acknowledgement

I thank the Centre for Development of Advanced Computing, Pune, for the three-month internship from 15 June 2026 to 15 September 2026, and Professor Pavan Dhote for guidance during the project. The system documented here answers questions from PDF evidence that has been extracted, chunked, embedded, and indexed, and it keeps the document and page identity of that evidence with the answer.

Abstract

A general-purpose language model can produce fluent text and still miss, or invent, a fact that exists only inside a particular PDF. This project builds a retrieval-augmented generation path that keeps the PDF as the source of the answer. A file is identified by the SHA-256 digest of its bytes. pypdf extracts each page. Cleaning removes control characters, rejoins hyphenated line breaks, and drops repeated running headers after the first page. A page with fewer than 15 non-space characters is marked for OCR. Tesseract is invoked only when PyMuPDF can render the page and the Tesseract binary is runnable; otherwise the page remains IMAGE_ONLY and the rest of the document continues. Blocks are classified as headings or paragraphs and packed into chunks of at most 1,200 characters with 100 characters of overlap. Short chunks in the same section are merged, and near-duplicates are removed with a SequenceMatcher threshold of 0.92.

Each accepted chunk keeps its document identifier, file name, section, and page numbers. The active embedding provider is the local Sentence Transformers model all-MiniLM-L6-v2, which emits 384-dimensional vectors. Elasticsearch 8.12.2 stores the text and the vector in the index rag_corpus_v1. The vector field is a dense_vector with cosine similarity and an HNSW index. Keyword search uses a BM25 match on the content field. Semantic search embeds the question with the same model and runs a k-nearest-neighbour query. The active hybrid ranker retrieves a pool of 20 from each method, min-max normalises each score list, and computes 0.3 times the normalised BM25 score plus 0.7 times the normalised vector score. Reciprocal rank fusion is coded and is not the configured strategy. The top five fused hits are formatted as context. On the command-line and demo path, a grounding gate abstains when there are no hits, when the leading score is below 0.35, or when a requested role is absent from the evidence. The language-model call uses temperature 0. The default provider in settings is a mock client. The five answers stored in docs/demo-results were produced by Llama 3.2 through Ollama.

The stored corpus contains 60 PDFs, 1,479 pages, 1,448 pages with recovered text, 31 pages still marked as requiring OCR, and no page-level extraction failures. Those PDFs produced 2,880 chunks, 2,880 extractive summaries, and 316 composite summaries. The live index held 12,054 documents: 6,076 from crawled PDFs and 5,978 from HTML. The embedding mapping was 384 dimensions, indexed, cosine. A pytest re-run reported 281 passed, 2 skipped, and 7 errors. The errors occur while deleting a temporary PDF that PyMuPDF still has open on Windows; the test bodies passed. No labelled retrieval dataset matched to this index has been scored, so precision, recall against gold passages, and NDCG remain pending.

Keywords

Retrieval-augmented generation; PDF extraction; pypdf; PyMuPDF; Tesseract OCR; section-aware chunking; Sentence Transformers; all-MiniLM-L6-v2; Elasticsearch; BM25; dense vector; HNSW; cosine similarity; min-max hybrid fusion; grounding; source provenance.

Table of Contents

The contents below are generated from the heading styles when the field is updated in Word.

Declaration 3
Acknowledgement 3
Abstract 3
Keywords 4
Table of Contents 4
Manual chapter order 7
List of Figures 7
List of Tables 7
List of Abbreviations 7
How to Read This Report 8
Part A — Problem, Objectives, and Innovation 9
Chapter 1. Introduction 9
1.1 What retrieval-augmented generation is 9
1.2 Why a conventional language model is not enough 9
1.3 The PDF problem this system is built for 9
1.4 Users and applications 9
1.5 Scope 9
1.6 Chatbot, search engine, and this system 9
Chapter 2. Problem Statement 9
2.1 The engineering problem 9
2.2 Difficulties the design responds to 10
2.3 What the problem statement does not include 10
Chapter 3. Objectives and Non-Objectives 10
3.1 Objectives 10
3.2 Non-objectives 11
Chapter 4. Innovation and Technical Novelty 11
4.1 Comparison 12
Chapter 5. Proposed Solution and System Overview 12
5.1 Stages 12
5.2 Data flow 14
5.3 A question walking through the pipeline 16

Part B — Technology Stack and Architecture 17
Chapter 6. Technology Stack 17
6.1 How each technology is used 17
6.2 Limits of the chosen stack 18
Chapter 7. Repository and Module Structure 18
Chapter 8. System Architecture and Data Flow 19
8.1 Layers 19
8.2 What is retained, and why 19
Part C — Detailed Module Implementation 21
Chapter 9. PDF Ingestion Pipeline 21
9.1 Purpose and problem 21
9.2 Input and output 21
9.3 Internal working 21
9.4 Identifiers, pages, and errors 21
9.5 Tests, status, limitations 22
Chapter 10. PDF Text Extraction 23
10.1 Purpose and problem 23
10.2 Libraries and page procedure 23
10.3 Images and the example 25
10.4 Tests, status, limitations 26
Chapter 11. Text Cleaning and Preprocessing 26
Chapter 12. Context-Aware Chunking 28
12.1 Why chunking exists 28
12.2 The algorithm that is actually used 28
12.3 Pseudocode 29
12.4 Example, configuration, and limits 30
Chapter 13. Metadata and Data Lineage 30
Chapter 14. Embedding Generation 32
14.1 What an embedding is 32
14.2 Providers, batches, and cosine 32
Chapter 15. Elasticsearch Knowledge Base 33
Chapter 16. BM25 Keyword Retrieval 34
Chapter 17. Vector Retrieval and Semantic Search 35
Chapter 18. HNSW Algorithm 36
Chapter 19. Hybrid Retrieval 37
19.1 Verified configuration 37
19.2 Min-max formula and a numerical illustration 38
Chapter 20. Ranking and Context Assembly 40

Chapter 21. LLM Integration and Response Generation 42
Chapter 22. Citation and Source Attribution 44
Chapter 23. End-to-End Query Execution 44
23.1 Passage retrieved for the worked PDF 45
Part D — Algorithms, Configuration, and Execution 47
Chapter 24. Algorithms and Pseudocode 47
1. PDF validation 47
2. Page-wise extraction 47
3. OCR decision 48
4. Text preprocessing 48
5. Chunk splitting 48
6. Overlap 48
7. Duplicate detection 48
8. Chunk merging 48
9. Embedding generation 49
10. Cosine similarity 49
11. BM25 retrieval 49
12. HNSW search 49
13. Min-max normalisation 49
14. Hybrid fusion 49
15. Top-k ranking 49
16. Context construction 50
17. Prompt assembly 50
18. LLM generation 50
19. Citation mapping 50
Chapter 25. Configuration Reference 50
Chapter 26. How to Run the Project 52
26.1 What the operator should expect 53
26.2 Troubleshooting 53
Chapter 27. Testing and Validation 53
Part E — Results, Evaluation, and Reliability 55
Chapter 28. Experimental Results 55
28.1 Saved answer pages 56
Chapter 29. Retrieval Evaluation 59
Chapter 30. Response Quality and Hallucination Control 60
Chapter 31. Performance and Scalability 60
Part F — Implementation Status and Future Scope 61
Chapter 32. Implementation Status Matrix 61

Chapter 33. Limitations 62
33.1 Observed 62
33.2 Structural, and not yet measured 62
Chapter 34. Future Scope and Proposed Enhancements 62
Chapter 35. Conclusion 63
Part G — Appendices and Technical References 65
Chapter 36. Full Module Reference 65
Chapter 37. Sample Data and Execution Outputs 66
Chapter 38. Sample Query Walkthrough 67
Chapter 39. Diagram Index 67
Chapter 40. Setup and Command Reference 68
Chapter 41. References 68

Manual chapter order

- Front matter: Declaration, Acknowledgement, Abstract, Keywords, Contents, Figures, Tables, Abbreviations, How to read this report
- Part A. Problem, objectives, and innovation — Chapters 1 to 5
- Part B. Technology stack and architecture — Chapters 6 to 8
- Part C. Module implementation — Chapters 9 to 23
- Part D. Algorithms, configuration, and execution — Chapters 24 to 27
- Part E. Results, evaluation, and reliability — Chapters 28 to 31
- Part F. Status and future scope — Chapters 32 to 35
- Part G. Appendices — Chapters 36 to 41

List of Figures

Each figure is numbered in the order it is inserted and is captioned under the image. Chapter 39 repeats that numbered list after every figure has been placed, so the numbers there match the body. The workflow sheets show the data entering and leaving each stage. The saved answer screenshots for the Aadhaar Data Vault, high-performance computing, ACTS, and cyber-forensics questions are included with the pipeline dashboard.

List of Tables

Each table is numbered in the order it is inserted and is captioned above the grid. The tables cover cover facts, objectives, the comparison with a chatbot and a vector-only index, the technology stack, module responsibilities, chunk metadata, the Elasticsearch mapping, retrieval settings, algorithms, configuration, commands, tests, corpus counts, the five saved answers, evaluation metrics, implementation status, proposed work, and the module inventory.

List of Abbreviations

Table 2. Abbreviations

Abbreviation	Meaning in this report
ANN	Approximate nearest neighbour
BM25	Best Matching 25 keyword ranking function
CLI	Command-line interface implemented by run.py
DPI	Dots per inch, used when a page is rendered for OCR
HNSW	Hierarchical Navigable Small World graph index
IDF	Inverse document frequency
kNN	k-nearest neighbours
LLM	Large language model
MRR	Mean reciprocal rank
NDCG	Normalised discounted cumulative gain; not implemented here
OCR	Optical character recognition
RAG	Retrieval-augmented generation
RRF	Reciprocal rank fusion; implemented, not the active strategy
TF	Term frequency

How to Read This Report

Read Chapters 1 to 5 for the problem and the stage model. Read Chapters 6 to 8 for the stack and the data that survives each stage. Read Chapters 9 to 23 when the question is how a particular module behaves, including its limits. Chapter 24 collects the algorithms. Chapter 25 is the configuration reference. Chapter 26 is the run book. Chapters 28 and 38 use the files in docs/demo-results and should be read together with Chapter 22, because the saved answers contain two citation layers.

The repository also contains a website crawler and a knowledge-graph store. They share the index and they appear in the saved demo because that demo called answer_question with include_kg left true and the source filter left at all. This report specifies the PDF path. It records the shared index and the demo exactly, and it does not restate the crawler design.

Two entry points exist. python run.py ingest, ingest-all, process-pdfs, and index go through rag.ingestion and later embed chunks together with extractive summaries. PDFRAGPipeline.ingest_pdf embeds and can index the chunks during the same call, and it does not build those summaries. The live index contains chunk, chunk_summary, and composite_summary documents, which is the shape produced by the CLI index command.

Part A — Problem, Objectives, and Innovation

Chapter 1. Introduction

1.1 What retrieval-augmented generation is

Retrieval-augmented generation answers a question in two stages. A retriever first selects a small set of passages from an external store. A generator then writes the answer from those passages. The language model is still used, and the passages constrain what it is allowed to rely on. In this project the external store is an Elasticsearch index of PDF chunks, and the generator is either the mock client or a chat-completions endpoint.

1.2 Why a conventional language model is not enough

A model trained on a general corpus does not contain the bytes of a PDF that was supplied after training. It also has a finite context window, so a long PDF cannot be pasted into every question. If the model answers from memory, a page number and a document name are not available to check. The engineering response in this repository is to extract the PDF, cut it into provenance-bearing chunks, retrieve a few of them, and pass only those chunks to the model.

1.3 The PDF problem this system is built for

A lengthy PDF mixes headings, paragraphs, running headers, page numbers, and sometimes scanned pages or embedded images. A keyword search over the whole file returns a hit without a stable page citation. A single vector index over the whole file loses local terminology. The pipeline therefore works page by page, keeps page numbers on every text chunk, and searches both the words and the embedding.

1.4 Users and applications

The intended user is an engineer or reviewer who has a set of PDFs and needs an answer that can be traced to a file and a page. The command-line ask command, the localhost POST /ask endpoint, and PDFRAGPipeline.answer_question are the three ways to submit that question. The stored demonstration asks institutional questions and returns the source list beside the prose.

1.5 Scope

The scope of this report is the path from a PDF path to a RAGAnswer: validation, extraction, cleaning, chunking, metadata, embeddings, indexing, BM25, vector search, hybrid fusion, context assembly, generation, and the source list. Website acquisition and knowledge-graph construction are outside the detailed chapters. They are mentioned where the saved demo and the shared index require an accurate reading.

1.6 Chatbot, search engine, and this system

A conventional chatbot generates a reply from the conversation and from the model. It does not open the PDF at question time. A document search engine returns ranked passages and leaves the reader to compose the answer. This system does both jobs and then keeps them separate: the retriever returns passages with scores and page numbers, and the generator writes a reply that the grounding rules try to keep inside those passages. The source list is returned even when the model abstains.

Chapter 2. Problem Statement

2.1 The engineering problem

Given a directory of PDFs and a natural-language question, return a short answer whose supporting passages can be identified by document and page, and abstain when the retrieved evidence does not clear the configured gate. The solution has to survive empty pages, repeated headers, chunks that would otherwise split a sentence, and questions whose wording does not repeat the PDF's wording.

2.2 Difficulties the design responds to

- Long PDFs cannot be placed entirely in a model context for every question.
- Page text from pypdf contains hyphenation, control characters, and running headers that should not become retrieval terms.
- A scanned page yields almost no characters. The code detects that condition and can call Tesseract; it also continues when Tesseract is absent.
- Keyword search ranks exact terminology and misses a paraphrase. Vector search does the reverse and can miss a rare identifier.
- BM25 scores and cosine scores are not on the same scale, so they cannot be added until each list is normalised.
- A fluent answer can still be unsupported. The ask path therefore checks hit count, a minimum score, and a small set of role strings before the model is called.
- An answer without a chunk identifier cannot be audited. Chunk identifiers and page numbers are stored on the text chunk and copied into RAGAnswer.sources.

2.3 What the problem statement does not include

The problem statement does not include rewriting the question, remembering earlier turns, reranking with a cross-encoder, or reading tables as structured data. Those items are listed as future work in Chapter 34 because the code does not implement them.

Chapter 3. Objectives and Non-Objectives

3.1 Objectives

Table 3. Objectives and the evidence that fixes their status

Objective	Status	Evidence
Accept a PDF path and reject missing, unreadable, corrupt, or empty files	DONE	PDFProcessor.process_pdf; extraction tests
Identify a document by the SHA-256 of its bytes	DONE	generate_document_id
Extract text page by page with pypdf	DONE	PDFProcessor; 1,479 stored pages
Clean whitespace, hyphenation, and repeated headers	DONE	clean_extracted_text, remove_repeated_boilerplate
Detect sparse pages and attempt OCR	PARTIAL	check_requires_ocr and OCRProcessor; host Tesseract was not on PATH during the test re-run
Save embedded images with PyMuPDF	PARTIAL	extract_page_images; stored image_chunks_generated summed to 0
Chunk by headings with overlap, merge, and near-duplicate removal	DONE	Chunker.chunk_structurally; 2,880 PDF chunks
Keep page numbers on text chunks	DONE	ChunkModel.page_numbers; demo PDF hit lists page 1

Keep page numbers on summary records	PARTIAL	summary_to_indexable does not copy page numbers; demo summary hit has an empty page list
Generate 384-dimensional embeddings	DONE	LocalEmbeddingProvider; live mapping dims 384
Index text and vectors in Elasticsearch	DONE	ElasticsearchIndexer; 12,054 documents
Retrieve with BM25	DONE	BM25Retriever
Retrieve with cosine kNN	DONE	VectorRetriever; dense_vector similarity cosine
Fuse with min-max weights 0.3 and 0.7	DONE	fuse_hybrid_results; settings.yaml
Offer reciprocal rank fusion	DONE as code, inactive	compute_rrf_scores; fusion_strategy is min_max
Assemble context and call an LLM at temperature 0	DONE	answer_question; demo answers
Abstain on empty, weak, or role-mismatched evidence	DONE on the ask path	grounding_failure; RAGPipeline.answer_query does not apply the score gate
Return a structured source list	DONE	RAGAnswer.sources in q1.json to q5.json
Score a labelled gold set with NDCG	PENDING	metrics.py has hit rate, recall, and MRR only; no scored run is stored

3.2 Non-objectives

The following are outside the implemented PDF path: multi-turn memory, query rewriting, cross-encoder reranking, a new embedding model, agent orchestration, and a gold question set aligned to the indexed document identifiers. LLM-assisted chunk organisation exists in Chunker.chunk_with_llm and is off, because use_llm_assisted_organization defaults to false and settings.yaml does not enable it. The cyber-forensics artifact records llm_calls_made as 0.

Chapter 4. Innovation and Technical Novelty

The system is more than a script that pastes a PDF into a chat window. The behaviours below are present in the source. Each one is limited to what the code actually does.

- The ingestion path is modular: extraction, chunking, embedding, indexing, retrieval, and generation are separate classes with a shared AppConfig.
- A text chunk carries the pages of the blocks it was built from, so a citation can name a page without guessing.
- Chunk boundaries prefer headings, blank lines, and sentence ends, and a 100-character tail is repeated so a split sentence remains visible in the next chunk.
- The embedding provider is selected by configuration: local, mock, or a custom HTTP endpoint. The active value is local.
- Keyword and semantic retrieval run independently and are fused in application code, so the weights can be inspected and tested without changing Elasticsearch.
- The vector field uses Elasticsearch HNSW because index is true on the dense_vector. The project sets num_candidates and leaves m and ef_construction at the engine default.
- Figure, table, and equation numbers are parsed with regular expressions and can boost a BM25 hit or a fused score when the question names the same number.

- The ask path can refuse to call the model. The refusal sentence is fixed: "The information is not available in the provided context."
- Tests can run with the mock embedding provider and the mock language-model client, so retrieval fusion and prompt text can be checked without a network model.

4.1 Comparison

Table 4. Traditional PDF chatbot, conventional vector RAG, and this project

Capability	PDF chatbot	Vector-only RAG	This project
Reads the supplied PDF	Sometimes, as one prompt	After an external index is built	Yes, through PDFProcessor and Chunker
Keeps page numbers	Usually no	Only if the index schema has them	Yes on text chunks
Keyword search	No	No	BM25 on the content field
Semantic search	No	Yes	Cosine kNN, 384 dimensions
Score fusion	No	No	Min-max, 0.3 / 0.7, pool of 20
Abstains before generation	No	Rarely	Yes on the ask path, threshold 0.35
Returns chunk identifiers	No	Depends on the wrapper	Yes, RAGAnswer.sources
OCR for a sparse page	Rare	Rare	Implemented; binary was absent in the test re-run

Chapter 5. Proposed Solution and System Overview

5.1 Stages

The running system follows thirteen stages. The class or function that owns each stage is named so the later chapters can be read against this list.

- 1. PDF ingestion — ingest_uploaded_pdf, process_pdf, or PDFRAGPipeline.ingest_pdf.
- 2. Extraction — PDFProcessor.process_pdf using pypdf.
- 3. Validation and cleaning — clean_extracted_text and remove_repeated_boilerplate.
- 4. Context-aware chunking — Chunker.chunk_structurally, then _merge_small_chunks.
- 5. Provenance — ChunkModel fields plus lineage written by rag.ingestion.process_pdf.
- 6. Embeddings — LocalEmbeddingProvider.embed_documents.
- 7. Indexing — ElasticsearchIndexer.index_records.
- 8. BM25 — BM25Retriever.search.
- 9. Vector search — VectorRetriever.search.
- 10. Hybrid ranking — HybridRetriever.search and fuse_hybrid_results.
- 11. Context — build_rag_context.
- 12. Generation — get_llm_client and generate_completion at temperature 0.
- 13. Attribution — build_source_references, returned on RAGAnswer.sources.

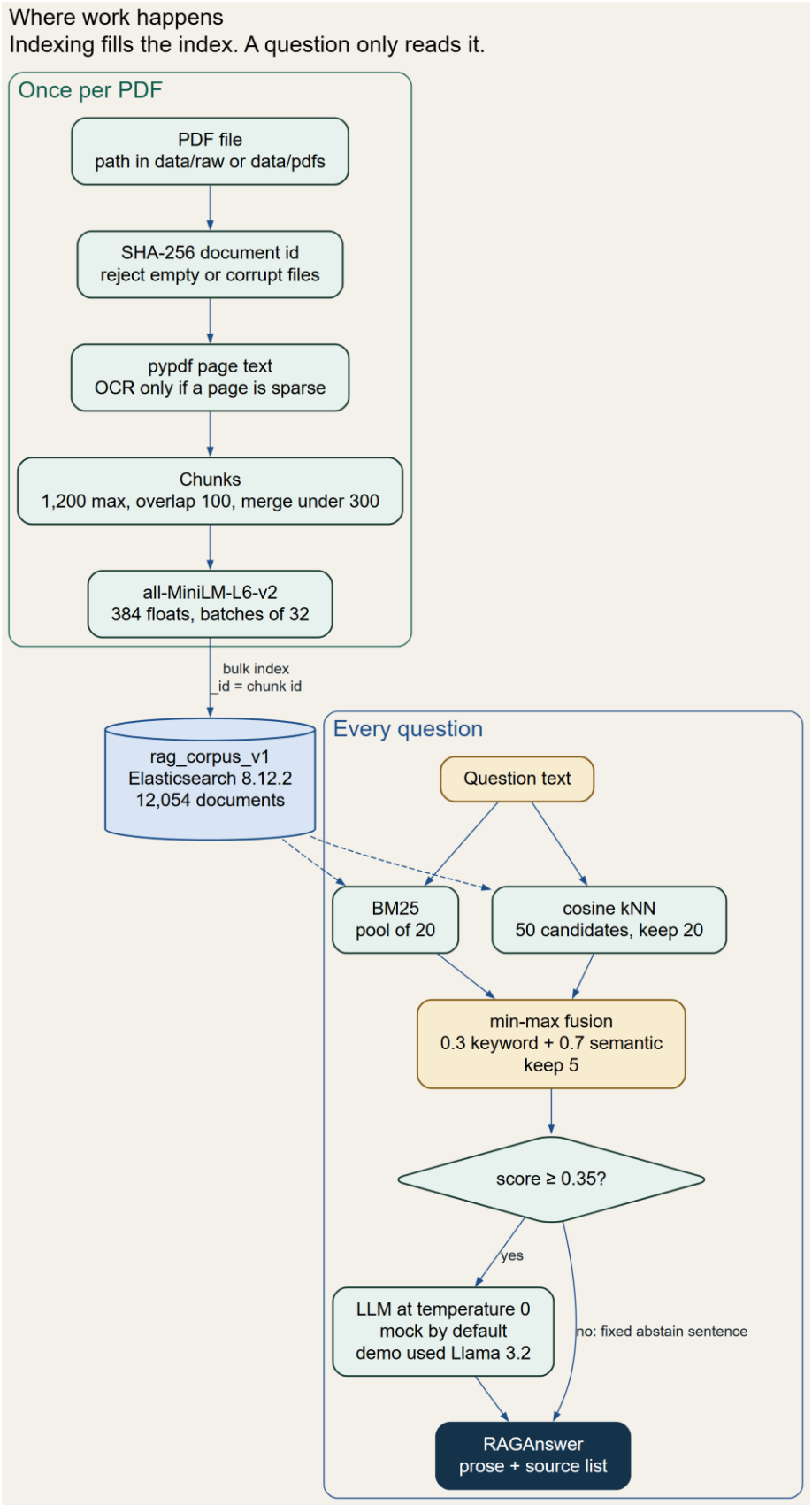


Figure 1. System layers of the PDF question-answering path

The diagram reads from the bottom. The input layer accepts a path. Extraction and chunking produce text with page numbers. The embedding layer turns that text into 384 floats. The index stores the text and the floats. Retrieval and ranking select a few records. Generation writes the reply, and the response layer returns the reply together with the source objects.

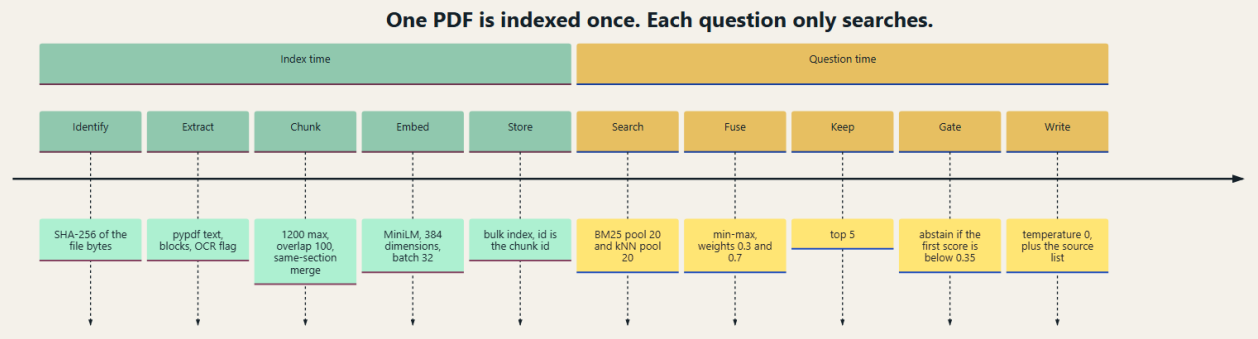


Figure 2. End-to-end path from a PDF file to a grounded answer

Each box is one stage boundary. A failure to read or parse the PDF raises an exception for that file. A sparse page does not stop the file. An unreachable Elasticsearch cluster skips indexing inside PDFRAGPipeline and is reported on ElasticsearchStatus; the CLI index command raises when the cluster cannot be used. An empty or weak retrieval on the ask path returns the fixed abstain sentence and does not call the model.

5.2 Data flow

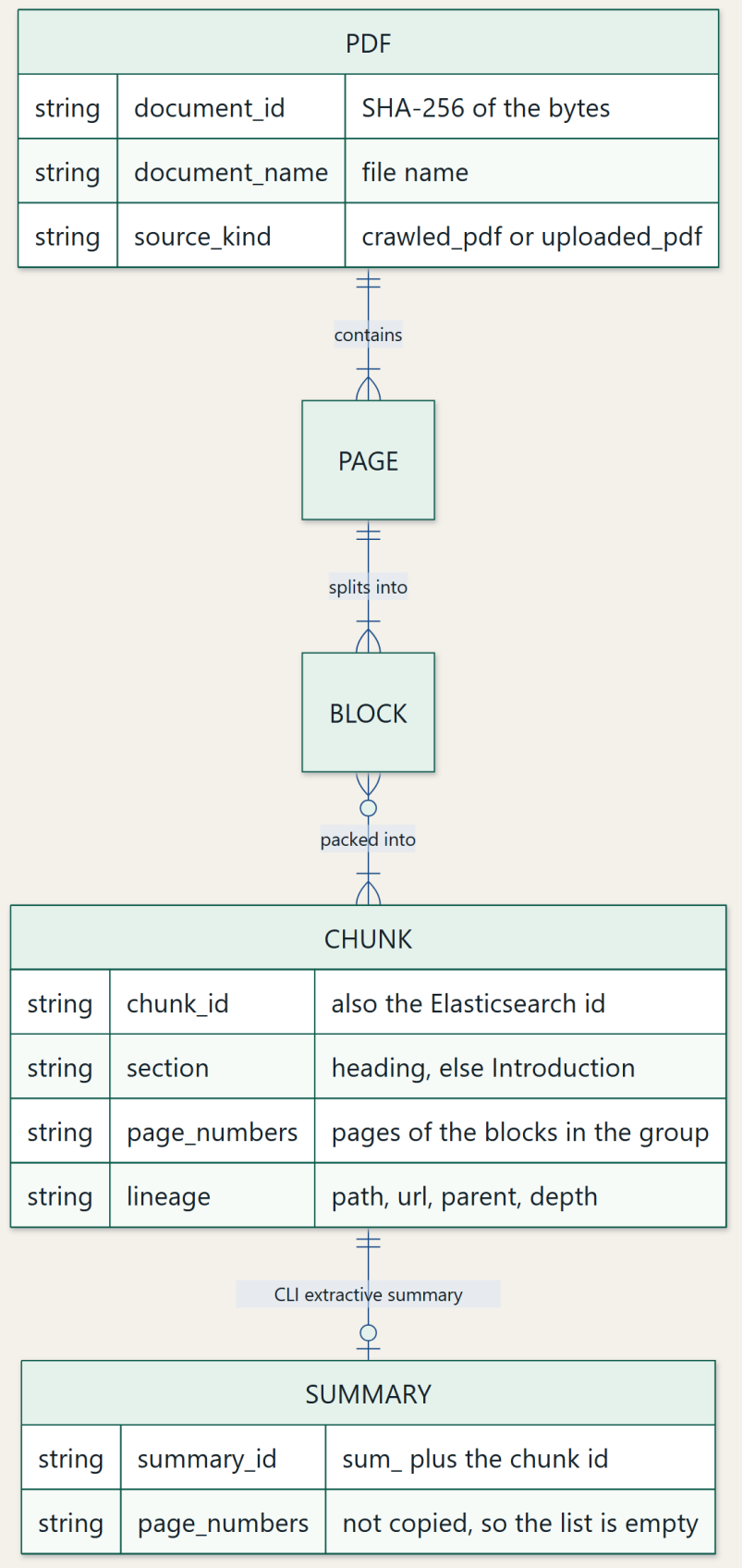


Figure 3. Data lineage retained from the file to the index

The bytes determine the document identifier. The file name becomes document_name. Each chunk receives a chunk_id of the form doc_<document_id>_chunk_<index>. Page numbers are the sorted set of block pages in the group. The index document uses the chunk identifier as its _id and stores the same page list on text chunks. Summaries are separate documents and, in the current mapper, do not carry that page list.

5.3 A question walking through the pipeline

The stored question "What cyber forensics work does C-DAC describe?" is the running example; Chapter 38 expands it. The PDF 82f38954477f2c53.pdf was extracted to 25 pages and 61 chunks with zero LLM chunking calls. Hybrid retrieval placed chunk 002 of that document, covering page 1, in the fused list with final score 0.6678. The ask call also attached five knowledge-graph rows because the demo script passed include_kg implicitly through its default. Llama 3.2 then wrote the answer stored in docs/demo-results/q5.json. The application source list, not the model's paragraph, is the page-level citation.

Part B — Technology Stack and Architecture

Chapter 6. Technology Stack

Versions below were read from the project virtual environment and, for Elasticsearch, from the cluster at http://localhost:9200. A library is listed because it is imported by the PDF path or pinned in requirements.txt and present in that environment.

Table 5. Technology stack present in the environment used for this report

Technology	Version observed	Role	Module
Python	3.12.10	Runtime	Entire repository; Docker image python:3.12-slim-bookworm
pypdf	5.9.0	Page text and PDF metadata	rag/extract/pdf_processor.py
PyMuPDF	2.1	Render a page for OCR and extract embedded images	rag/extract/ocr_processor.py
pytesseract	0.3.13	Python binding to the Tesseract binary	OCRPageContext.ocr_page_text
Tesseract binary	Not on PATH in the test re-run	OCR engine	Called only when ocr_dependencies_available is true
Pillow	13.0.0	Open rendered page images	ocr_processor.py
sentence-transformers	3.4.1	Local embeddings	LocalEmbeddingProvider
all-MiniLM-L6-v2	Configured name	384-dimensional encoder	embedding.model in settings.yaml
NumPy	2.5.4	Vector arrays inside the embedding library and the HNSW benchmark	embedder.py uses the model; hnsw_benchmark.py uses NumPy directly
Elasticsearch	server 8.12.2; client 8.19.3	BM25, kNN, and storage	rag/index/indexer.py and rag/retrieve
Pydantic	2.14.3	Configuration and record models	rag/config.py, rag/models.py
PyYAML	6.0.3	Read config/settings.yaml	load_config
python-dotenv	1.2.4	Load a .env file without printing it	load_config
pytest	8.4.2	Unit and contract tests	tests/
requests	pinned >=2.31,<3	Custom embedding and LLM HTTP calls	embedder.py, llm_client.py
Docker Compose	docker-compose.yml	Elasticsearch 8.12.2 service and optional app/UI	repository root

6.1 How each technology is used

Python hosts every stage. pypdf is the text extractor: PdfReader.pages[i].extract_text supplies the raw page string, and reader.metadata supplies the document information dictionary. PyMuPDF is opened only for rendering and image extraction. The import name in the source is fitz. On PyMuPDF 2.1 that import emits a

deprecation warning, which the test re-run recorded. pytesseract calls the external tesseract binary. The Python package can be installed while the binary is missing; OCRProcessor then logs a warning and leaves sparse pages unchanged. The Docker image installs the tesseract-ocr package, so a container build has the binary that the host test run lacked.

Sentence Transformers loads all-MiniLM-L6-v2 and checks that its dimension equals embedding_dimension, which is 384. The encode call does not set normalize_embeddings in this code. Cosine similarity is declared on the Elasticsearch field, and Elasticsearch normalises during that similarity. NumPy is a dependency of the model stack and is used directly by the synthetic HNSW benchmark to build unit vectors and a brute-force dot product.

Elasticsearch stores the corpus and executes both BM25 and kNN. The project does not reimplement BM25 or HNSW. It sends a match query and a knn clause. Pydantic validates configuration, including the rule that keyword_weight plus semantic_weight equals 1. PyYAML reads the settings file, and environment variables override fields by upper-case name. Secrets are held as SecretStr and masked by get_masked_dict. pytest collected the suite re-run for this report. requests is used only when a custom embedding or LLM endpoint is configured.

6.2 Limits of the chosen stack

pypdf reads the text layer and does not reconstruct a visual layout. PyMuPDF image extraction saves raster images; it does not understand a chart. MiniLM is a compact English encoder, so very long technical strings are represented by a single 384-dimensional vector per chunk. Elasticsearch must be running for live retrieval. The mock LLM returns a fixed sentence and cannot be used as evidence of answer quality. The five stored answers are the evidence for a live model, and they came from Llama 3.2 through Ollama as recorded in responses.md.

Chapter 7. Repository and Module Structure

The repository root contains run.py, requirements.txt, config/settings.yaml, docker-compose.yml, Dockerfile, the rag package, tests, data, and docs. data/raw is the upload directory. data/pdfs holds PDFs with a manifest. data/extracted holds one JSON artifact per processed document. docs/demo-results holds the five saved answers, their HTML rendering, and screenshots.

Table 6. Repository responsibilities for the PDF question-answering path

Path	Responsibility	Status
run.py	CLI: status, ingest, index, query, ask, ui, evaluate	DONE
rag/config.py	YAML plus environment configuration and validation	DONE
rag/models.py	Chunk, page, document, and statistic models	DONE
rag/ingestion.py	PDF artifact build and corpus indexing	DONE; this path filled the live index
rag/pdf_rag_pipeline.py	Programmatic ingest, embed, optional index, and answer	DONE; does not write summaries
rag/extract/pdf_processor.py	pypdf extraction and cleaning	DONE
rag/extract/ocr_processor.py	Optional OCR and image extraction	PARTIAL until Tesseract is present
rag/extract/reference_extractor.py	Figure, table, and equation references	DONE
rag/index/pdf_chunker.py	PDF chunking	DONE

rag/index/embedder.py	Mock, local, and custom embedding providers	Local provider is active
rag/index/indexer.py	Mapping and bulk indexing	DONE
rag/index/summary_builder.py	Extractive summaries of chunks	DONE on the CLI path
rag/retrieve/bm25.py	BM25 retriever	DONE
rag/retrieve/vector.py	kNN retriever	DONE
rag/retrieve/hybrid.py	Fusion and hybrid retriever	DONE
rag/retrieve/factory.py	Select bm25, vector, or hybrid	DONE
rag/answer/pipeline.py	Context, prompts, and RAGAnswer	DONE
rag/answer/grounding.py	Abstain rules	DONE on the ask path
rag/answer/llm_client.py	Mock and custom chat client	Mock is the settings default
rag/services.py	search_corpus, search_kg, answer_question	DONE
rag/ui.py	Localhost inspector and POST /ask	DONE as code
rag/eval/metrics.py	hit@k, recall@k, reciprocal rank	DONE; no stored scored run
tests/	Unit and contract tests	281 passed and 7 teardown errors

Configuration is isolated in AppConfig. Ingestion orchestrates extraction and chunking. Processing lives under rag/extract. Embeddings, storage, retrieval, and generation are separate packages. Tests mirror those packages and also include contract tests for the shared index schema. A count of Python modules under rag/, excluding caches, is 52 files. Fourteen of those files are the PDF extraction, chunking, retrieval, and answer path named in the table. The crawl and graph packages are additional modules in the same tree.

Chapter 8. System Architecture and Data Flow

8.1 Layers

The input layer is a filesystem path. The document-processing layer yields a DocumentModel whose pages contain text, blocks, an extraction status, and an OCR flag. The chunking layer yields ChunkModel objects and a ChunkingStatistics record. The embedding layer yields one float vector per text. The storage layer writes IndexableRecord documents. The retrieval layer returns SearchResult or HybridSearchResult objects. The ranking layer is inside hybrid fusion: min-max, weights, optional boosts, and a cut to top_k. The generation layer builds a prompt and calls the client. The response layer is the RAGAnswer model: query, answer, abstained, abstain_reason, sources, retrieval_mode, and chunks_retrieved.

8.2 What is retained, and why

- Original file path and byte hash, so the same PDF can be recognised and the source file can be opened.
- Page text and blocks, so chunking can follow the reading order of the extractor.
- Chunk text, section, and page numbers, so retrieval can quote a location.
- The embedding, so a later question can be compared without calling the encoder on every stored chunk.
- The fused score and the source list, so a reviewer can see which passages supported the call.

The original PDF bytes remain in data/pdfs or data/raw. The index does not store the PDF binary. Image files, when extraction succeeds, are written under data/extracted/images. The stored PDF statistics report zero image chunks, so image OCR text did not enter the chunk list for that corpus.

Part C — Detailed Module Implementation

Each module chapter uses the same headings: purpose, problem, input, output, internal working, functions, algorithm, configuration, dependencies, execution flow, example, diagram, tests, status, and limitations. Later chapters refer back instead of repeating a mechanism.

Chapter 9. PDF Ingestion Pipeline

9.1 Purpose and problem

Ingestion turns a PDF path into a stored artifact that the indexer can read. It solves the problem of accepting one file or a directory, rejecting files that are not readable PDFs, and recording which files failed without discarding the files that succeeded.

9.2 Input and output

Input is a path, or the directory data/raw for uploads and data/pdfs for files already recorded by the PDF download store. Output of process_pdf is a PageContentArtifact written as JSON. Output of the later index step is an Elasticsearch bulk result. PDFRAGPipeline.ingest_pdf returns an IngestResult that already includes embedding counts and an indexing result.

9.3 Internal working

ingest_uploaded_pdf checks that the file exists, copies it into data/raw when it is not already there, and calls process_pdf with source_kind uploaded_pdf. process_crawled_pdfs lists the download manifest and calls process_pdf with source_kind crawled_pdf. process_pdf runs PDFProcessor, then Chunker, copies lineage into every chunk, builds extractive summaries and composites, and writes the artifact. index_corpus loads every artifact selected by kb.source, embeds every record's text, and bulk-indexes them. The live kb.source is all.

Batch failure handling in PDFRAGPipeline.ingest_all_pdfs catches an exception per file and returns it in failures while keeping the successes. discover_pdf_paths lists *.pdf in the directory and does not recurse.

9.4 Identifiers, pages, and errors

The document identifier is computed before parsing, from the raw bytes. Page count comes from len(reader.pages) after pypdf opens the file. A missing file raises FileNotFoundError. A read error raises OSError. An unreadable PDF raises PdfReadError. A zero-page PDF raises ValueError. A failure inside a single page is caught, stored as extraction_status FAILED on that page, and does not abort the remaining pages. Multiple documents are a loop over paths. Original bytes stay at the source path; the upload helper additionally copies them into data/raw.

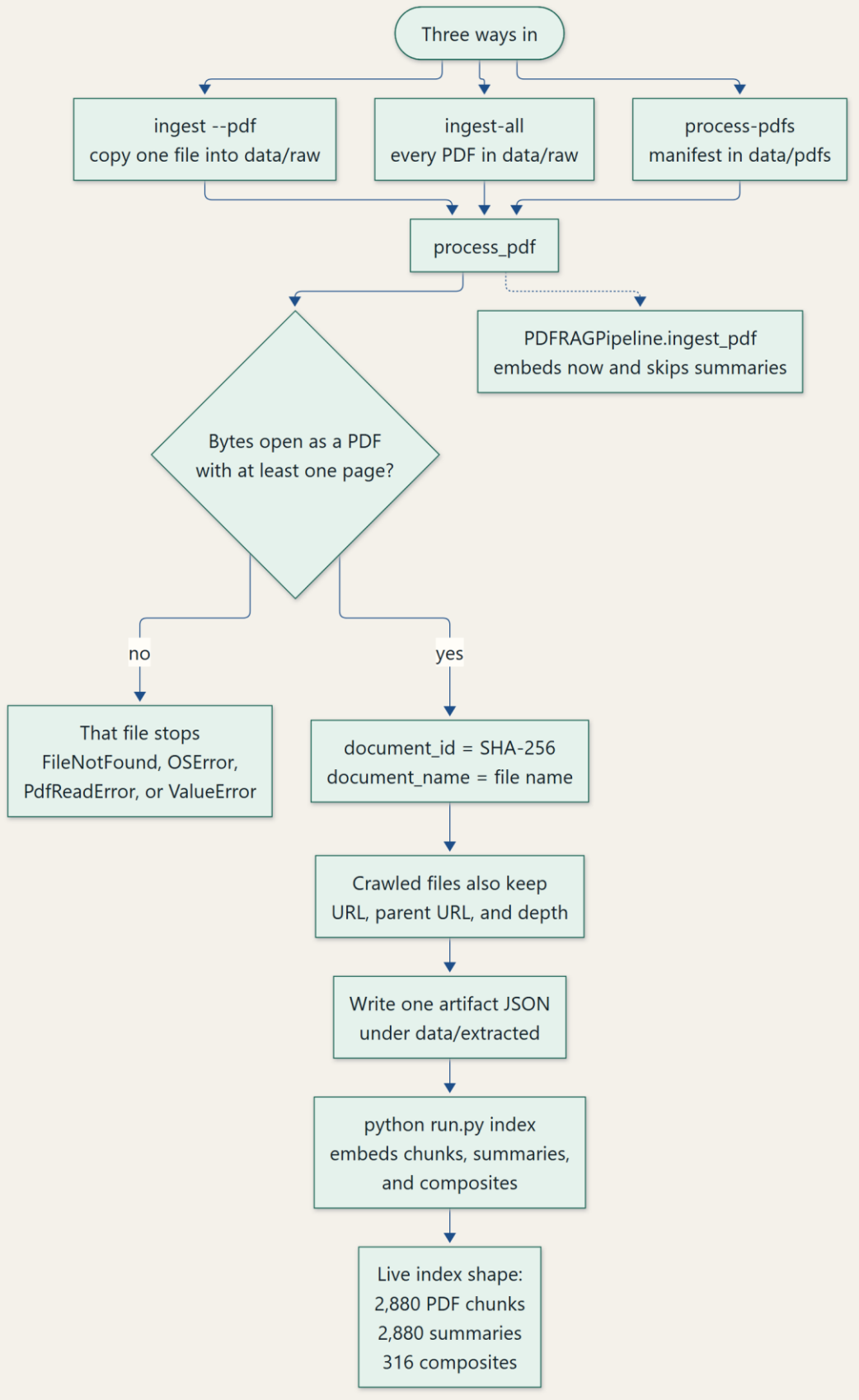


Figure 4. How a PDF enters the indexed corpus

9.5 Tests, status, limitations

test_port_pdf_rag_pipeline.py exercises the programmatic pipeline with stand-ins. test_port_crawler_pdf_pipeline.py and the extraction tests cover the processor used underneath. Status: DONE for the CLI artifact path, which matches the live index. Limitation: two ingest implementations exist,

and only the CLI path writes summaries. PDFRAGPipeline also forces source_kind to uploaded_pdf after chunking.

Chapter 10. PDF Text Extraction

10.1 Purpose and problem

Extraction converts a PDF page into text and blocks while remembering the page number. It has to distinguish a normal text page from a page whose text layer is almost empty, and it has to keep going when one page throws.

10.2 Libraries and page procedure

pypdf reads the text layer in the order the library returns from extract_text. The project does not add a second geometric sort. Blank lines split paragraphs. detect_block_type marks a short block as a heading when it matches a numbered heading, a known section word, an all-caps line, or a short line ending in a colon. Otherwise the block is a paragraph. Tables and lists are not parsed as structures; their text, if the text layer contains it, becomes ordinary blocks. Figure and table numbers are recognised later by regular expressions, which is reference detection rather than table reconstruction.

A text-based PDF yields a long string from extract_text. A scanned page typically yields an empty or tiny string. check_requires_ocr returns true when the non-space length is below 15. The page status becomes IMAGE_ONLY if OCR does not replace that text, and SUCCESS if text is present or OCR recovery succeeds. IMAGE_ONLY increments pages_requiring_ocr. It does not increment failed_pages. Document status is SUCCESS when failed_pages is zero, even if some pages are IMAGE_ONLY. That is why all 60 stored PDF artifacts have extraction_status SUCCESS while 31 pages remain OCR-flagged.

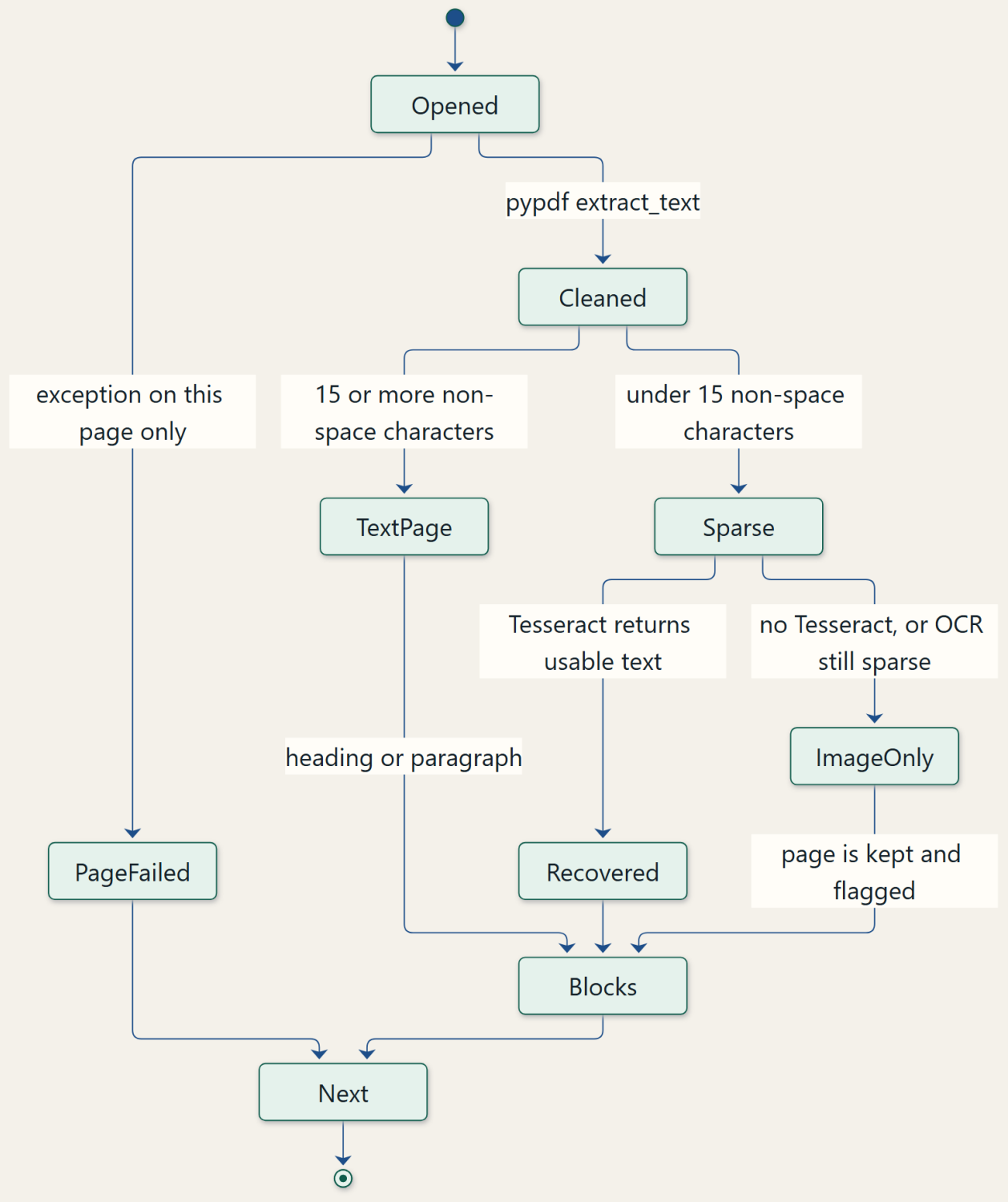


Figure 5. Page-wise text extraction

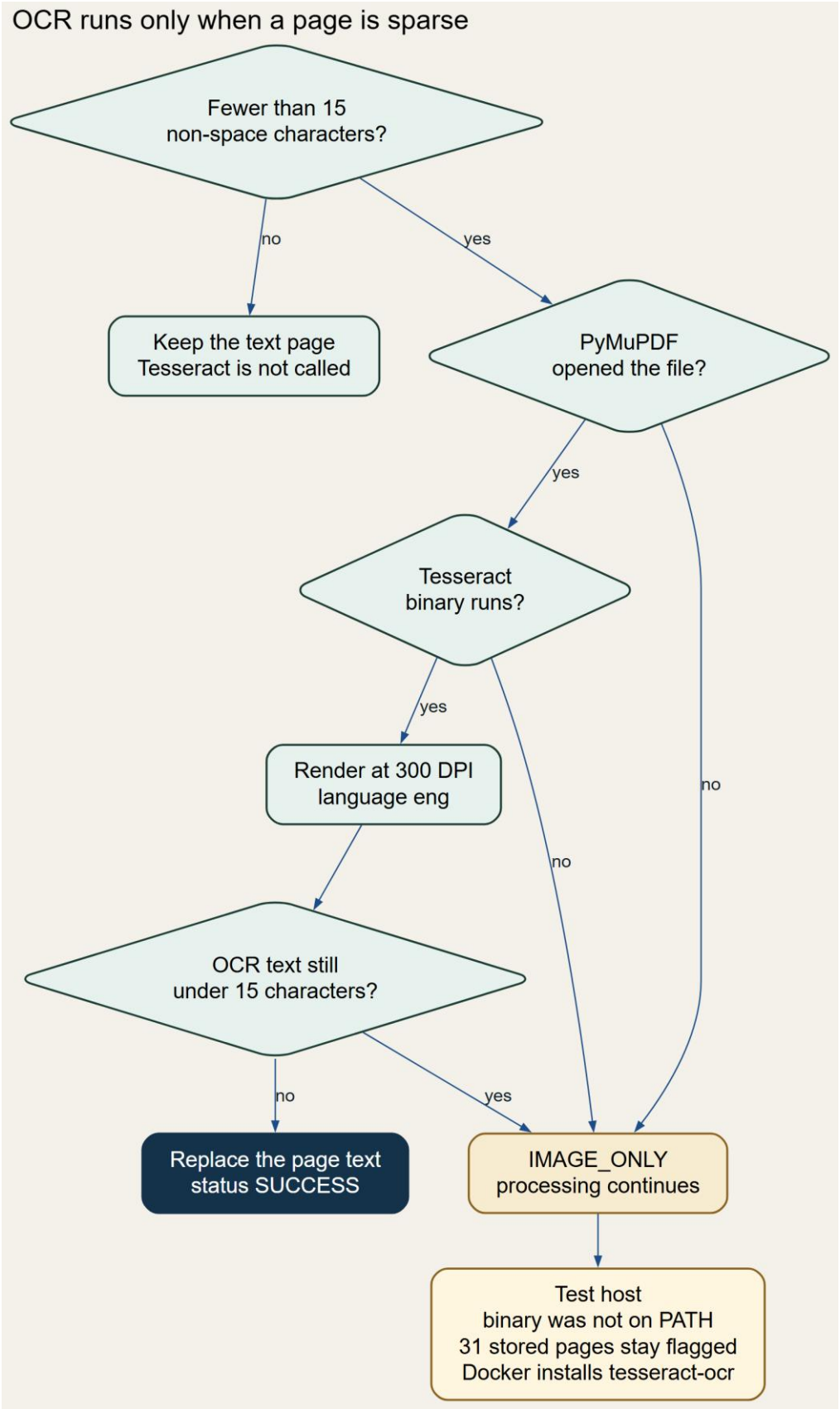


Figure 6. OCR decision for a sparse page

10.3 Images and the example

When image_extraction_enabled is true and PyMuPDF opens the file, each embedded image at least 50 pixels on both sides is written under data/extracted/images/<document_id>. OCR on that image runs only if

Tesseract is available. The 60 stored chunking statistics sum image_chunks_generated to 0, so no image produced a chunk of at least 15 OCR characters in that run.

Example, from the stored artifact of 82f38954477f2c53.pdf. The file has 25 pages, 25 successfully extracted pages, 0 OCR-flagged pages, and extraction duration 0.700 seconds. The first chunk, still on page 1, begins: "Invitation for Expression of Interest for Technology Reselling of Cyber Forensics Tools". That string is the extracted text after cleaning, not a reconstruction.

10.4 Tests, status, limitations

tests/test_pdf_extraction.py and tests/test_port_pdf_pdf_processor.py cover multi-page extraction, empty pages, and per-page failure. Their bodies passed and their teardowns raised WinError 32 because the temporary PDF was still open. Status of the extractor: DONE. OCR recovery: NEEDS LIVE VALIDATION on a host where tesseract is on PATH. Limitation: reading order is whatever pypdf returns, and a multi-column page can interleave columns.

Chapter 11. Text Cleaning and Preprocessing

clean_extracted_text drops characters below U+0020 except tab and newline, converts carriage returns, rejoins a hyphen that splits a word across a newline when the next fragment starts with a lowercase letter, collapses spaces and tabs, strips each line, and reduces three or more newlines to a blank line. remove_repeated_boilerplate then drops, on every page after page 1, lines that look like page numbers, short course-code patterns, labelled name or roll-number lines, or a two-to-four-word title-case running head. Page 1 is left intact so a title is not removed. is_boilerplate_block applies a similar majority vote to a whole block and is used again when blocks are created.

Empty blocks are skipped. Exact duplicate chunk text is removed later by a SHA-256 set, and near-duplicates are removed by SequenceMatcher against the last five accepted chunks. There is no separate language detector. Cleaning changes retrieval because running headers would otherwise dominate BM25, and a hyphenated word would miss the query term.

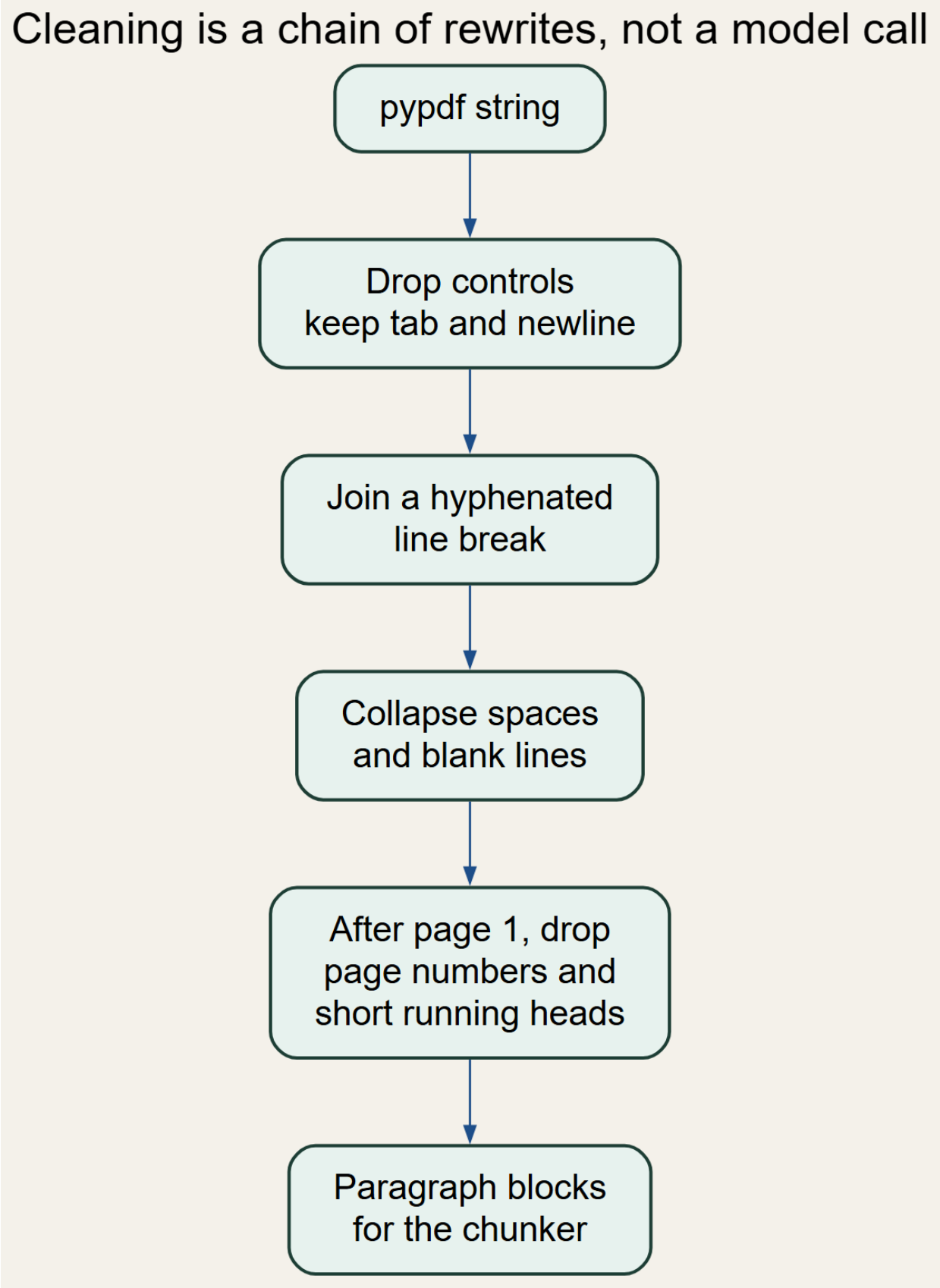


Figure 7. Text cleaning before chunking

Status: DONE. Tests cover extraction output rather than every regular expression in isolation. Limitation: a genuine short title-case line after page 1 can be removed by the running-head rule, and a header that does not match the patterns is kept.

Chapter 12. Context-Aware Chunking

12.1 Why chunking exists

An embedding model and a keyword index both work better on a passage than on an entire document, and the language-model context can hold only a few passages. The chunk is the unit of retrieval and the unit of citation.

12.2 The algorithm that is actually used

The active algorithm is structural, not a recursive character splitter and not a semantic embedding splitter. Blocks are walked in order. A heading closes the current group and becomes the section name. The initial section name, before any heading, is the string Introduction. A group is emitted when adding the next block would exceed 1,200 characters. If overlap is positive, the last 100 characters of the emitted text, trimmed to a word boundary, become a carry-over block on the next group. If the combined text still exceeds 1,200 characters, split_text_with_overlap cuts it. The cut prefers a blank line, then a newline, then a sentence ending, inside the window, and the next window starts overlap characters before the cut.

Validation rejects an empty chunk, a chunk shorter than min(20, min_chunk_size) characters, a missing identifier, a missing page, or a page number that is not in the document. Exact hash duplicates and near-duplicates at or above 0.92 similarity are counted as duplicates. Remaining chunks shorter than 300 characters are merged forward into the next chunk of the same section when the combined length is at most 1,200 times 1.3, which is 1,560. A short final chunk merges backward under the same section rule. Identifiers are renumbered after the merge. Image OCR chunks are appended after that, and reference metadata is added last. LLM chunking is a separate method and is not selected by the current configuration.

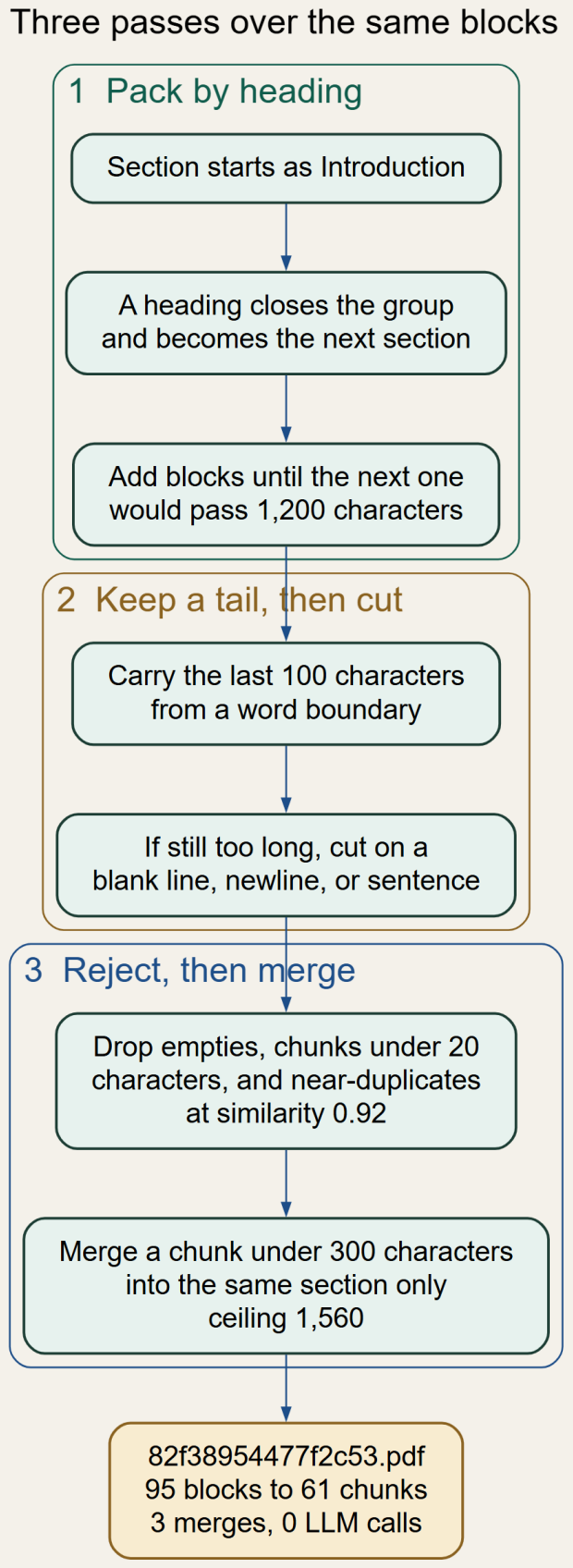


Figure 8. Structural chunking with overlap and merge

12.3 Pseudocode

```
section := "Introduction"; group := empty
for block in blocks:
    if block is a heading:
        emit(group, section); group := empty; section := block.text
```

```
    else if group would exceed 1200 characters:
        emit(group, section); group := overlap_tail(group, 100) + [block]
    else:
        group.append(block)
emit(group, section)
reject invalid and near-duplicate chunks
merge same-section chunks shorter than 300 characters
append image chunks whose OCR text has at least 15 characters
```

12.4 Example, configuration, and limits

Before chunking, 82f38954477f2c53.pdf had 25 pages and 95 blocks. After structural chunking it had 61 chunks, average length 842 characters, minimum 89, maximum 1,339, no multi-page chunks, 3 merges, 0 duplicates, 0 rejections, and 0 LLM calls. The maximum above 1,200 is allowed by the merge ceiling of 1,560. Across all 60 PDFs the text-chunk lengths run from 49 to 1,481 characters, with mean 933.6 and median 1,073.

Configuration: pdf_max_chunk_size 1200, pdf_chunk_overlap 100, pdf_min_chunk_size 300, duplicate_similarity_threshold 0.92. Status: DONE. Tests: test_pdf_chunker.py and test_port_pdf_chunker.py. Limitation: a heading detector can fire on a short line that is not a heading, and the overlap is a character tail rather than a whole sentence. Chunks can still start or end in the middle of a thought when no boundary is found inside the window.

Chapter 13. Metadata and Data Lineage

Provenance is the ability to walk from a retrieved hit back to a file and a page. The text-chunk schema below is the one produced for PDFs.

Table 7. PDF chunk metadata

Field	Meaning	Where it is set
document_id	SHA-256 of the PDF bytes	generate_document_id
document_name	File name	DocumentModel.document_name
chunk_id	doc_<id>_chunk_<nnn>	Chunker.emit and the merge renumber
section	Current heading text, or Introduction	chunk_structurally
page_numbers	Sorted pages of the blocks in the group	emit
content	Chunk text	emit
source_kind	uploaded_pdf or crawled_pdf	process_pdf
source_url	URL when the file was downloaded; otherwise null	crawl record or argument
lineage	Path, pdf id, parent URL, depth, hash, download time	process_pdf
figure_refs, table_refs, equation_refs	Numbers mentioned in the chunk	enrich_reference_metadata
embedding	384 floats, stored only in Elasticsearch	index_records

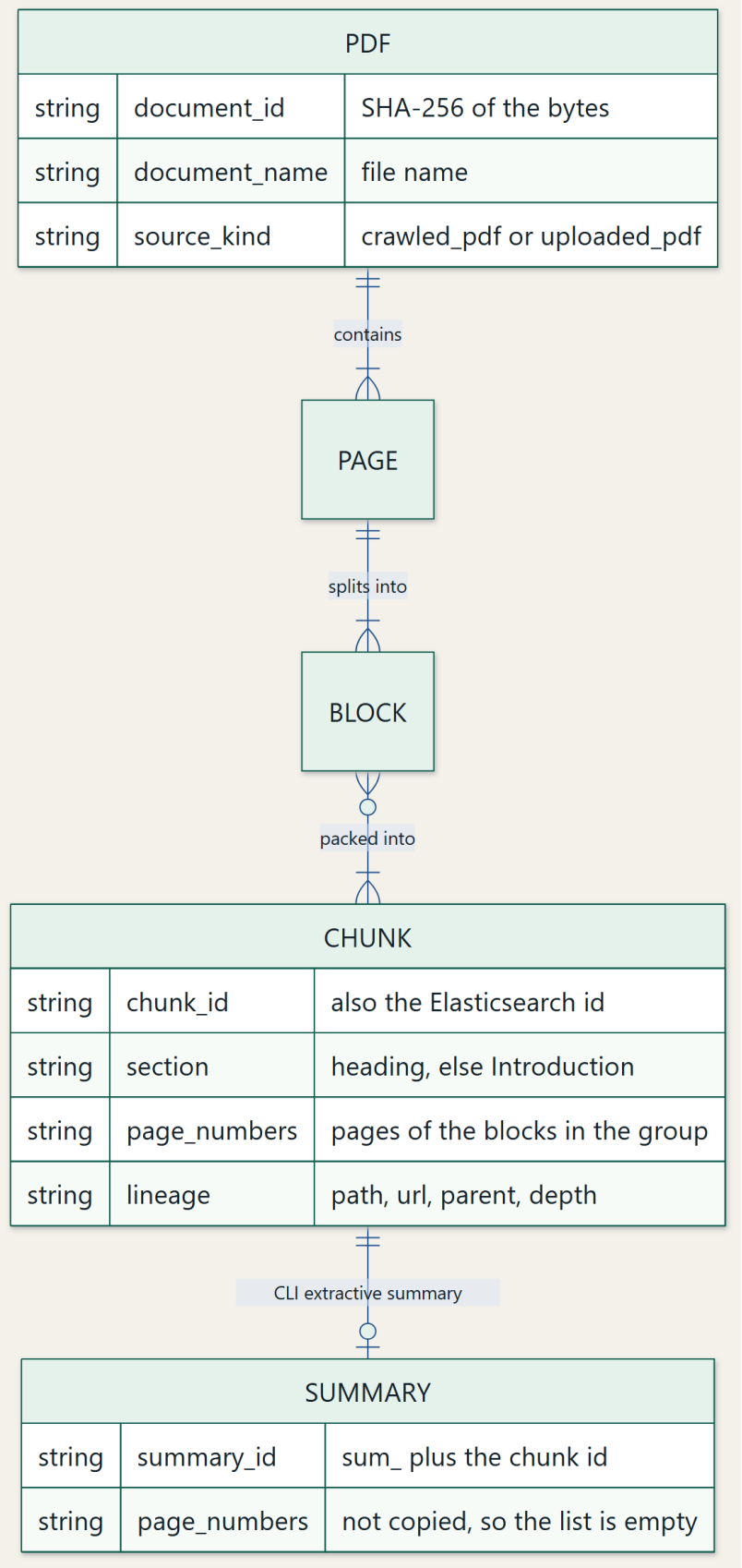


Figure 9. What stays attached to a PDF chunk

The demo hit for the cyber-forensics PDF shows why this matters. The chunk record carries page [1]. The summary record of the same chunk carries an empty page list, because SummaryModel has no page field and summary_to_indexable does not invent one. A citation that uses the summary hit cannot name a page unless it follows lineage.chunk_id back to the text chunk. Status: DONE for text chunks, PARTIAL for summaries.

Chapter 14. Embedding Generation

14.1 What an embedding is

An embedding is a list of real numbers that the encoder assigns to a piece of text. Pieces of text that the model considers similar receive vectors with a high cosine. The encoder is a neural network, all-MiniLM-L6-v2, loaded by Sentence Transformers. This project does not train it. The configured dimension is 384, and LocalEmbeddingProvider refuses to start if the loaded model reports a different dimension. The live index mapping confirms dims 384.

14.2 Providers, batches, and cosine

get_embedding_provider returns MockEmbeddingProvider when the provider is mock, LocalEmbeddingProvider when it is local, and CustomAPIEmbeddingProvider when it is custom_api or openai. Any other string falls through to the mock provider. The active settings value is local. Mock vectors are deterministic: a SHA-256 seed drives a Python random generator that fills 384 values in the range -1 to 1. They are for tests, not for meaning. The local provider encodes a query with encode(text) and a batch with encode(texts, batch_size=32). The code does not L2-normalise the list before storage.

Cosine similarity between vectors a and b is the dot product a·b divided by the product of the Euclidean norms. Elasticsearch computes it for the knn query because the field similarity is cosine. A one-dimensional illustration, not a real MiniLM vector: a = (1, 0) and b = (1, 1). The dot product is 1, the norms are 1 and sqrt(2), and the cosine is about 0.707. Equal vectors have cosine 1. The real vectors have 384 components; printing one would not make the geometry clearer, and none is copied into this report.

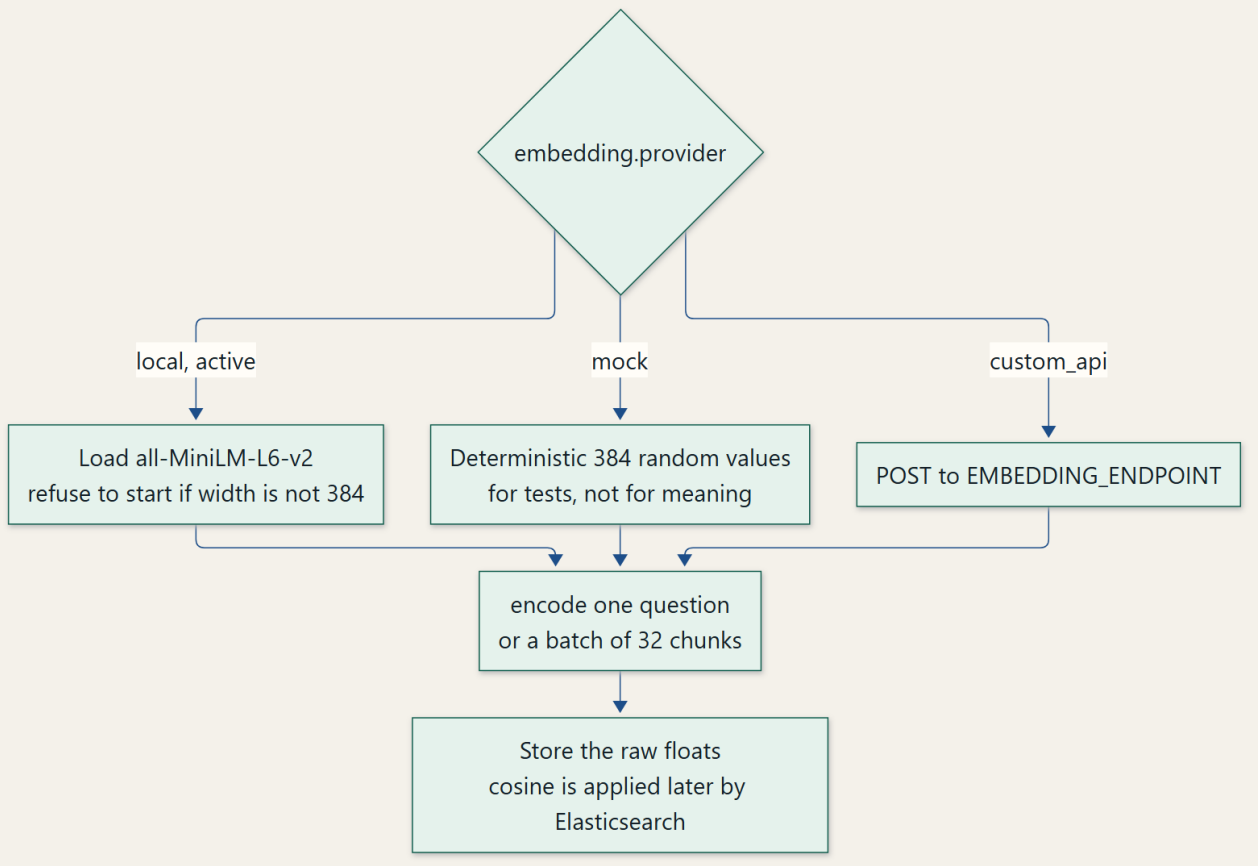


Figure 10. From chunk text to a 384-dimensional vector

Query embeddings use the same provider as document embeddings. A mock index cannot be searched sensibly with a local query vector, and the reverse is also true. Status: DONE for the local provider, verified by the mapping. Tests in test_port_pdf_embedder.py cover validation, the mock provider, and dimension

checks. Limitation: one vector represents the whole chunk, so a chunk that joins two topics is a compromise. The model is English-oriented.

Chapter 15. Elasticsearch Knowledge Base

Elasticsearch was selected because one index can run BM25 on a text field and approximate nearest-neighbour search on a dense_vector field, and because filters on keyword fields can restrict source_kind. The index name is rag_corpus_v1. create_index_if_not_exists writes the mapping only when the index is absent. If it exists, the code reads embedding.dims and raises when the value differs from 384.

Table 8. Elasticsearch fields that matter for a PDF chunk

Field	Mapping	Role
content	text	BM25 body
document_name, section	text plus keyword subfield	Display and exact filters
chunk_id, document_id, source_kind, content_kind	keyword	Identity and filters
page_numbers	integer	Citation pages
source_url, parent_url, depth	keyword or integer	Lineage for downloaded files
figure_refs, table_refs, equation_refs	keyword	Reference boosts
lineage, figure_pages, table_pages	object, enabled false	Stored, not indexed as text
embedding	dense_vector, dims 384, index true, similarity cosine	kNN
embedding_model, embedding_dimension, schema_version	keyword or integer	schema_version is the literal v3

index_records builds one bulk action per record. The _id is the record identifier, so reindexing the same chunk replaces it. The vector length is checked against 384 before the action is queued. delete_document_chunks issues a delete-by-query on document_id. There is no separate update API; a new bulk index of the same _id is the update. Persistence is the Elasticsearch data volume declared in docker-compose.yml as es_data. The cluster version was 8.12.2, the index existed, and the count was 12,054, of which 6,076 documents had source_kind crawled_pdf.

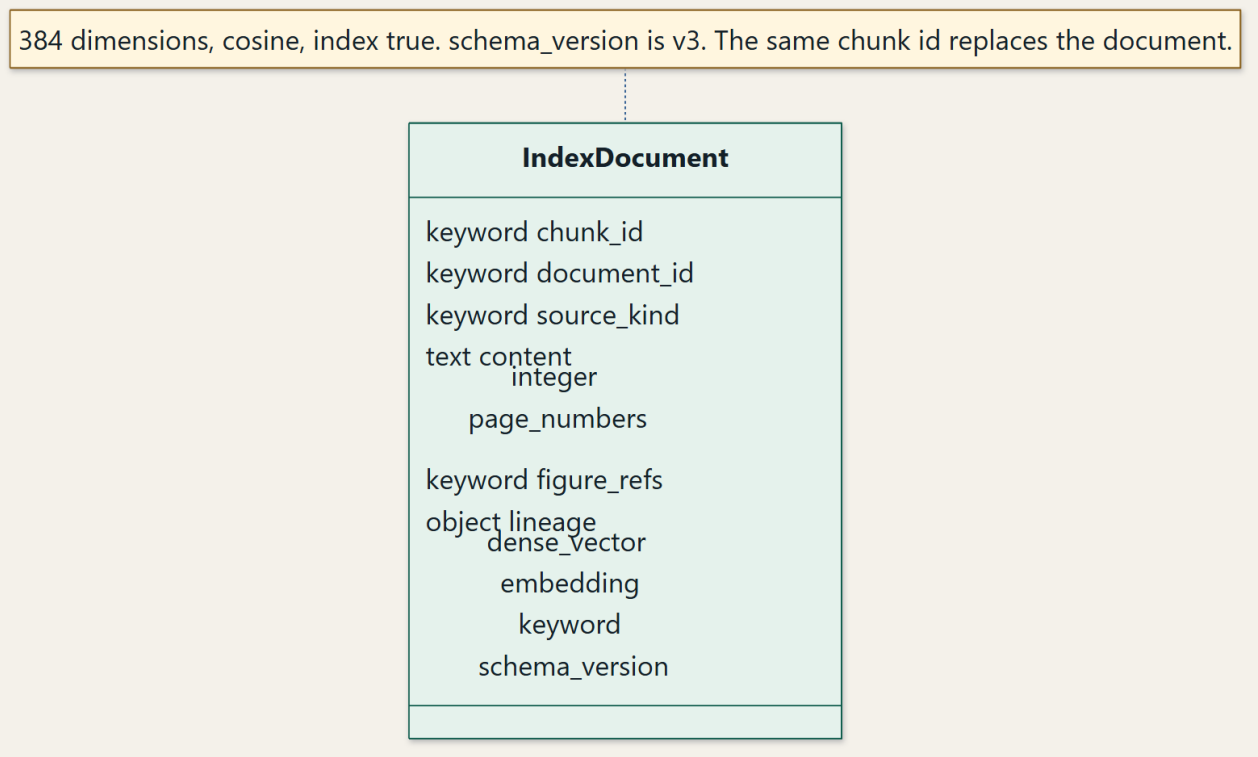


Figure 11. One chunk becomes one Elasticsearch document

A PDF text chunk and its Elasticsearch document are the same logical record: the chunk_id is both the model identifier and the document _id, and the embedding is added only in the index source. Status: DONE and verified live. Tests in test_port_pdf_es_indexer.py use a fake client. Limitation: the mapping is not updated in place when the code changes, and a dimension change requires a new index. The project does not set number_of_shards or a custom BM25 similarity.

Chapter 16. BM25 Keyword Retrieval

BM25 ranks a document for a query by combining three ideas. Term frequency is how often a query term occurs in the document, with diminishing returns. Inverse document frequency reduces the weight of a term that occurs in many documents. Document-length normalisation stops a long document from winning only because it contains more words. Elasticsearch implements this inside Lucene. The usual parameterisation, which this project does not override, is k1 = 1.2 and b = 0.75. The score of document D for query Q is the sum over query terms q of IDF(q) multiplied by (tf(q, D) · (k1 + 1)) / (tf(q, D) + k1 · (1 − b + b · |D| / avgdl)).

The query body built by BM25Retriever is a match on content. When the caller passes a source or a domain, bm25_query wraps that match in a bool query with a filter. Reference terms add should clauses on figure_refs, table_refs, or equation_refs with the configured boost, default 4.0. A visual question also adds a should clause on chunk_type image with boost 2.0. A small example of the IDF idea: if the token "forensics" occurs in 2 of 2,880 chunks and the token "the" occurs in nearly all of them, BM25 gives "forensics" the larger weight. The numbers in that sentence are an illustration of the formula, not a term statistic measured from the index.

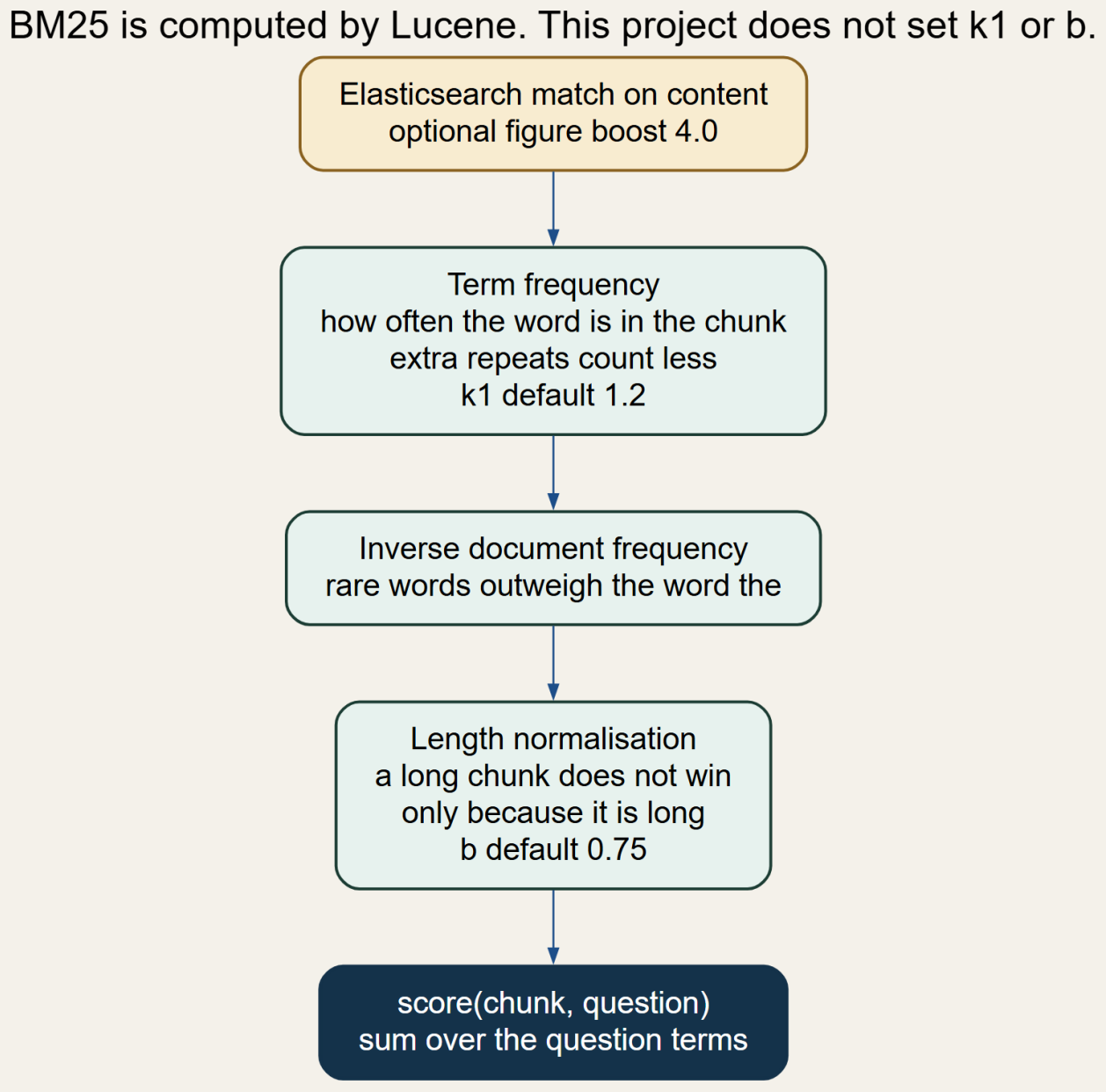


Figure 12. Keyword retrieval

BM25 is strong when the question uses the PDF's own names, identifiers, and section titles. It misses a paraphrase that shares no tokens. Status: DONE. Tests: test_port_pdf_retrieval_bm25.py. Limitation: the project uses the engine default parameters and does not expose k1 or b in settings.

Chapter 17. Vector Retrieval and Semantic Search

Vector retrieval embeds the question with the same model that embedded the chunks, then asks Elasticsearch for the k nearest stored vectors. The request sets k and num_candidates. num_candidates must be at least k; the configured values are a final k of 5 and 50 candidates, and the hybrid caller raises the result count to the candidate pool of 20 before fusion. The score returned by Elasticsearch for a cosine dense_vector is the similarity used to rank neighbours. A question that says "tools for investigating digital evidence" can still meet a chunk that says "cyber forensics tools" when the encoder places those phrases near each other, even though the token overlap is small. That is the difference from Chapter 16. The stored demo does not record the vector score separately from the fused score, so this report does not invent a cosine for that question.

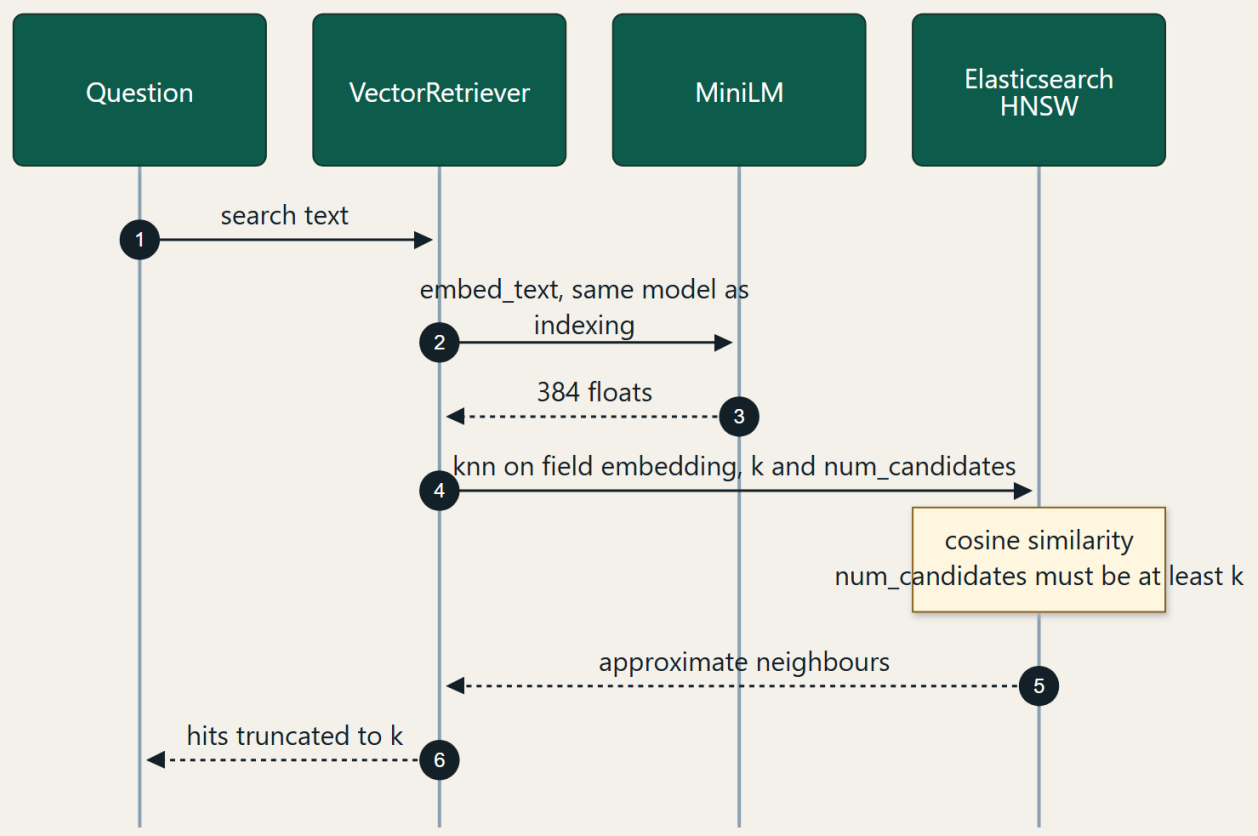


Figure 13. Semantic retrieval

Status: DONE as a query path, and the field it depends on was present in the live mapping. Tests: test_port_pdf_retrieval_vector.py. Limitation: nearest neighbours can be topically close and still fail to answer the question. The grounding threshold is the project's check on that risk.

Chapter 18. HNSW Algorithm

Comparing a query vector with every stored vector is exact and becomes expensive as the corpus grows. Hierarchical Navigable Small World graphs are an approximate nearest-neighbour index. Each vector is a node. Nodes are linked to nearby nodes. A few long-range links are placed in upper layers, and the base layer contains every node. Search enters at the top layer, moves greedily toward the query, and drops down a layer when the local neighbourhood stops improving. At the base layer it keeps a candidate set. The construction parameter m limits how many links a node keeps. ef_construction controls how widely the index looks while it is being built. A search-time parameter, often called ef_search, controls how many candidates are explored at query time. A larger value raises recall and latency.

This repository does not construct that graph in Python. Elasticsearch does, because the mapping sets embedding.index to true. The inspected mapping contains type, dims, index, and similarity. It does not contain index_options, so m and ef_construction are the Elasticsearch defaults rather than project settings. The project does set num_candidates, which is the search-time candidate count passed in the knn clause. The configured value is 50. rag/eval/hnsw_benchmark.py can compare a NumPy brute-force scan with an Elasticsearch kNN query on synthetic unit vectors. No output file from that benchmark is stored, so this report states no latency or recall number for HNSW.

```
Input: query vector q, graph, k, num_candidates
Enter the top layer at its entry node
While a neighbour is closer to q, move to it
Drop one layer and repeat
On the base layer, explore up to num_candidates nodes
Return the k closest nodes found
```

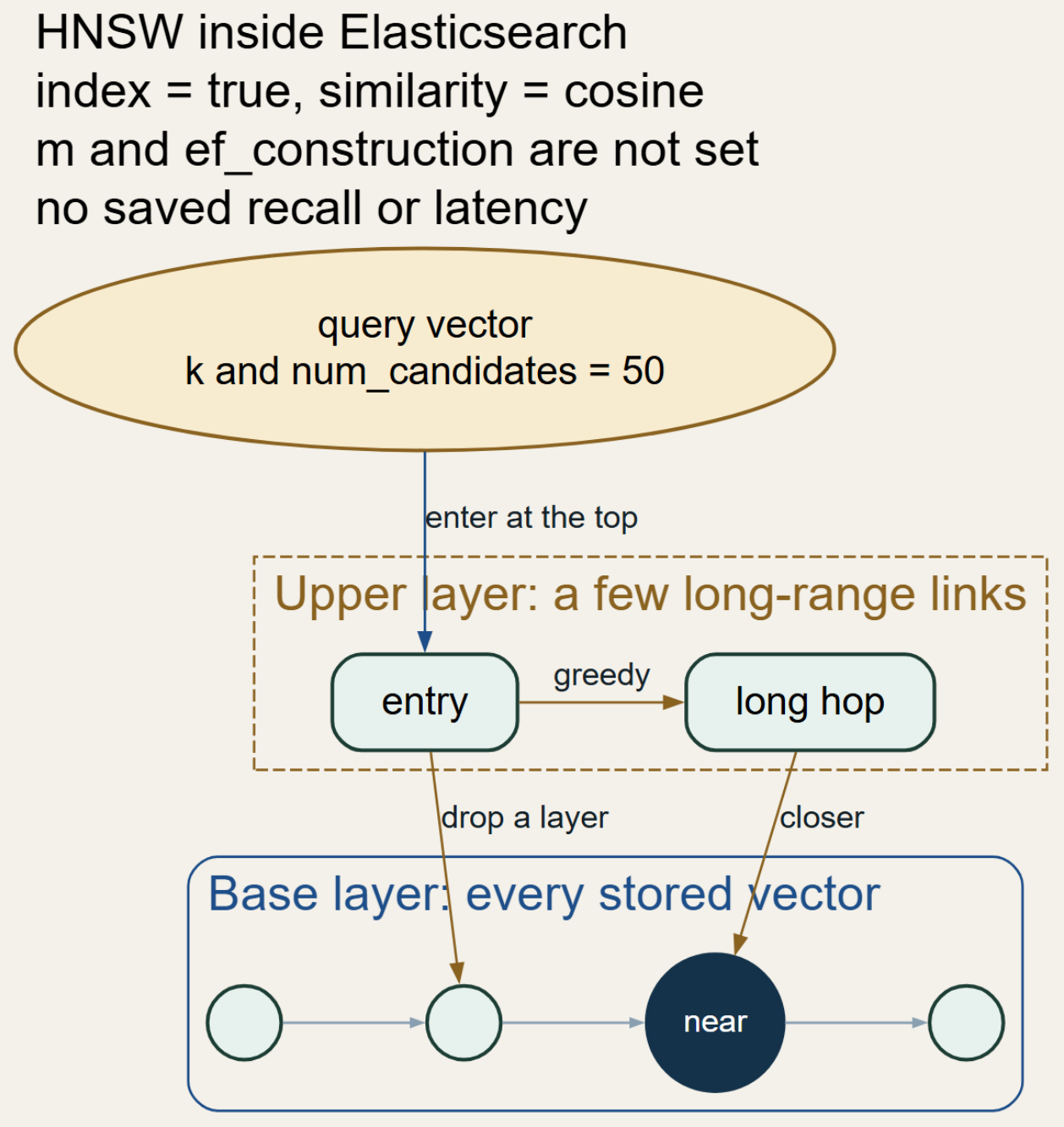


Figure 14. HNSW search, conceptual, matching the Elasticsearch dense_vector path

The pseudocode describes the engine algorithm. The project supplies q, k, and num_candidates. Status of the index setting: DONE. Status of measured HNSW recall: NEEDS LIVE VALIDATION. Complexity is sublinear in practice and is not given a project-specific big-O measurement here.

Chapter 19. Hybrid Retrieval

BM25 and vector search fail in different cases. The hybrid retriever runs both, then builds one ranked list. It is the default retrieval_mode.

19.1 Verified configuration

Table 9. Retrieval configuration verified against settings.yaml and the retriever code

Parameter	Value in settings and AppConfig	Used when

retrieval.mode	hybrid	get_retriever when the caller does not override it
top_k	5	Final cut after fusion
num_candidates	50	Vector search candidate count; raised to at least the pool
hybrid_candidate_pool	20	How many BM25 and vector hits are fetched before fusion
keyword_weight	0.3	Min-max fusion
semantic_weight	0.7	Min-max fusion
fusion_strategy	min_max	Active branch in fuse_hybrid_results
rrf_k	60	Only when fusion_strategy is rrf
figure_reference_boost	4.0	Added to the fused score when reference numbers overlap; also a BM25 should-boost
image_query_boost	2.0	Added for chunk_type image when the question looks visual

Changing keyword_weight toward 1 favours exact terminology. Changing semantic_weight toward 1 favours paraphrase. The two weights must sum to 1, and AppConfig rejects any other pair. Raising top_k puts more passages into the prompt. Raising hybrid_candidate_pool changes which scores are visible to min-max, because normalisation uses only the returned lists. Raising num_candidates asks HNSW to explore more nodes. rrf_k has no effect while the strategy remains min_max. Reciprocal rank fusion is implemented: for each list, a chunk at rank r contributes 1 / (rrf_k + r), and the two contributions are added. Selecting it requires fusion_strategy rrf.

19.2 Min-max formula and a numerical illustration

For a score list, the normalised value is (score − minimum) / (maximum − minimum). If the maximum equals the minimum, every score in that list becomes 1. A chunk missing from one list receives 0 for that side. The hybrid score is keyword_weight times the normalised BM25 score plus semantic_weight times the normalised vector score. With the active weights that is 0.3 · normBM25 + 0.7 · normVector.

Illustration, invented to show the arithmetic and labelled as such. BM25 scores A=12, B=6, C=9 normalise to 1, 0, and 0.5. Vector scores A=0.40, B=0.90, C=0.65 normalise to 0, 1, and 0.5. Hybrid scores are A=0.30, B=0.70, C=0.50. B ranks first because the semantic weight is larger. The saved demo scores are already fused; the JSON does not store the pre-normalisation BM25 and vector components, so those demo numbers are not forced into this example.

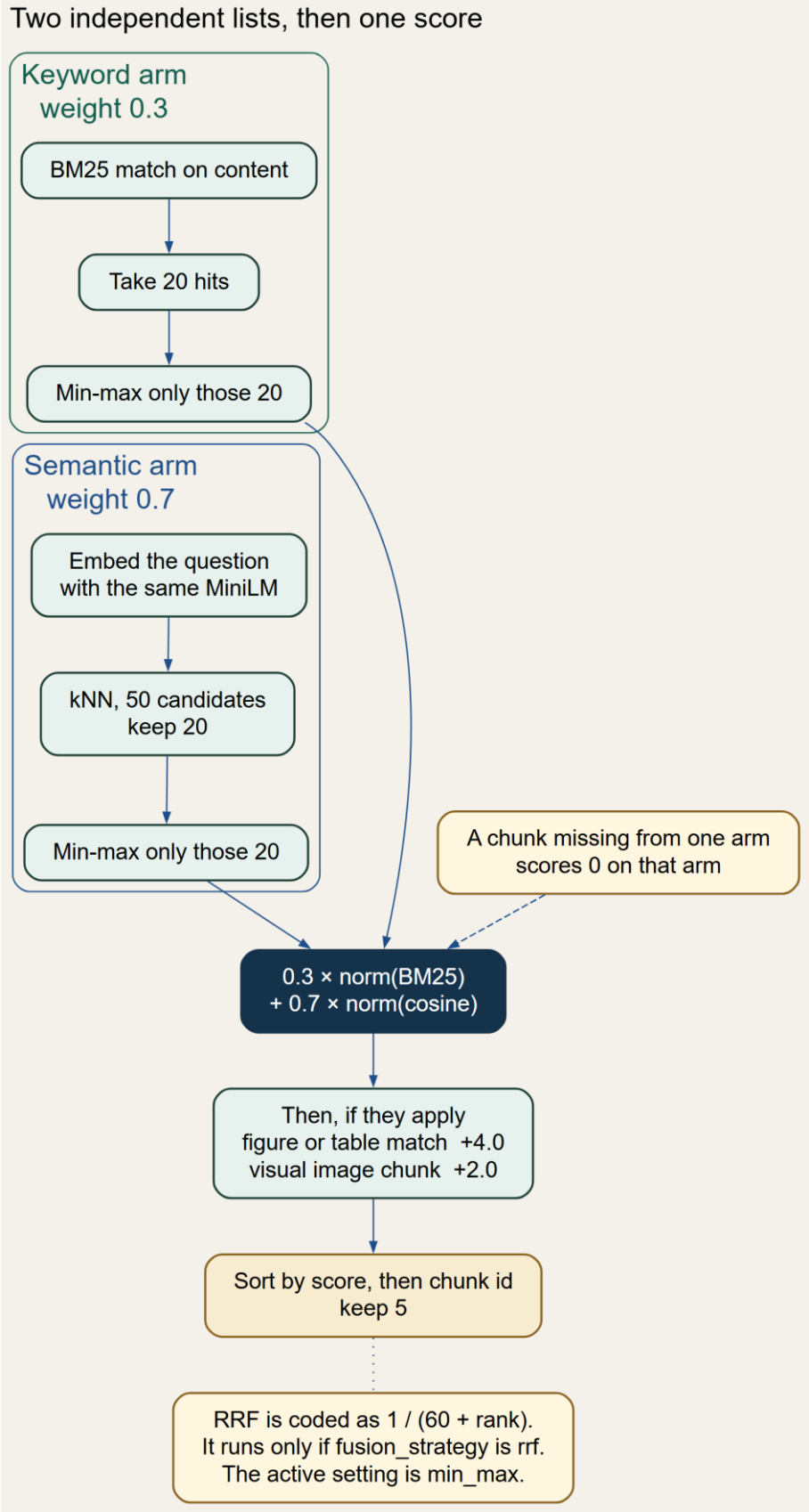


Figure 15. Hybrid retrieval and ranking

Invented scores, to show the arithmetic.
The demo JSON stores only the fused number.

Chunk	BM25	Norm	Vector	Norm	0.3 + 0.7
A	12	1	0.40	0	0.30
B	6	0	0.90	1	0.70 first
C	9	0.5	0.65	0.5	0.50

Figure 16. Illustrative min-max fusion with the active weights

Duplicate handling is by chunk identifier. merge_search_results keeps one object per chunk_id and records whichever of the two scores arrived. The final sort key is the fused score descending, then the chunk identifier ascending, and the list is cut to top_k. Status: DONE. Tests: test_port_pdf_retrieval_hybrid.py, 21 test functions, included in the 281 passes. Limitation: min-max on a pool of 20 is sensitive to one extreme score, and the weights are global rather than learned per question.

Chapter 20. Ranking and Context Assembly

After the two retrievers return, fusion is the ranker. There is no second model. Top-k filtering is the slice at the end of fuse_hybrid_results. Duplicate removal is the chunk_id merge described in Chapter 19. Context selection is "take the fused list in order". No token budget is implemented: build_rag_context concatenates every supplied hit. Metadata kept in the prompt block is source kind, document name, pages, figure and table references when present, section, chunk id, and the content text. Order is the list order, which is rank order for a hybrid result.

On the ask path the grounding gate runs before this formatting. If it fails, the model is not called and the context is never built. If it passes, and include_kg is true, graph hits are appended after the corpus hits and are formatted with their own provenance lines. The stored demo used that branch. The PDF-only formatting branch is the else clause in build_rag_context: document name and pages.

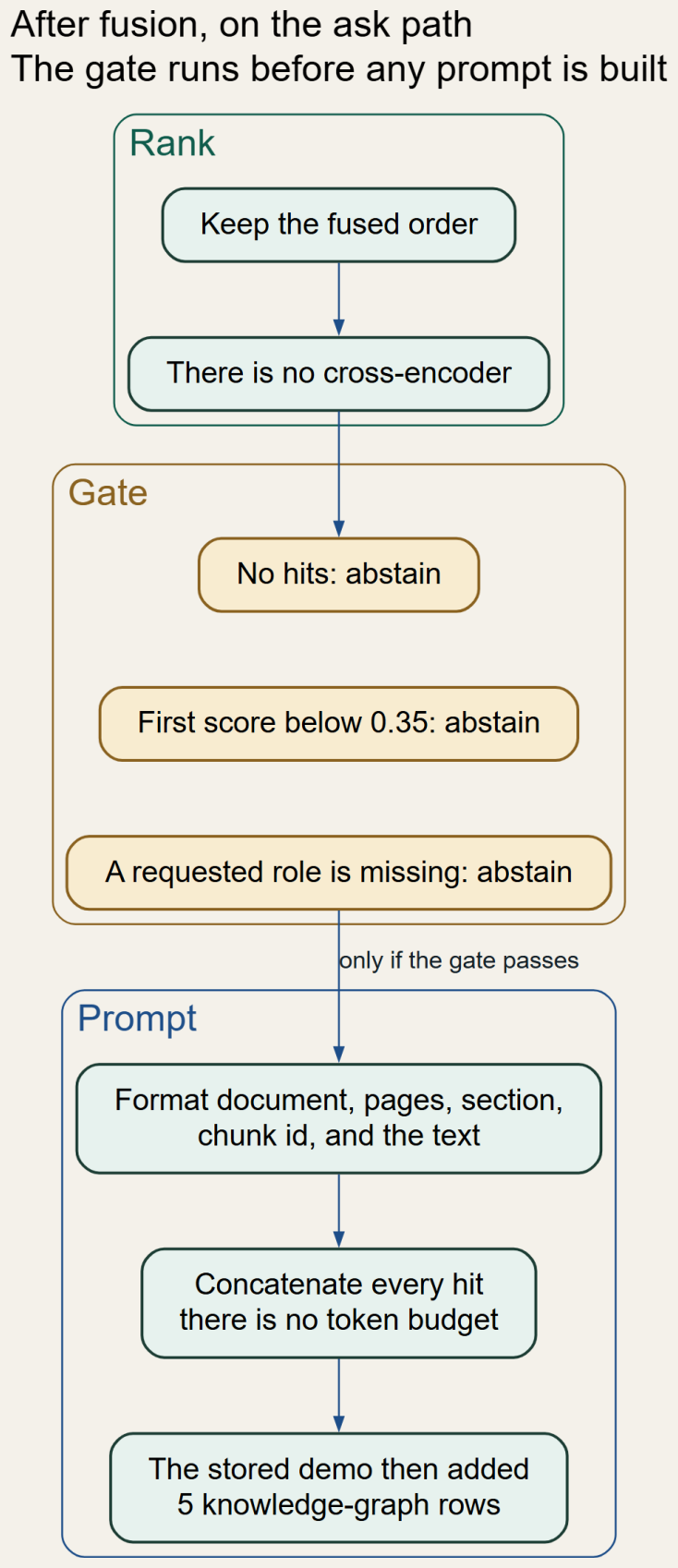


Figure 17. From ranked hits to the prompt context

Status: DONE for concatenation and metadata. PENDING for a token budget, because none exists. Limitation: five long chunks plus five graph rows can produce a large prompt, and a summary hit may be redundant with its parent chunk, as in the cyber-forensics demo where chunk 002 and its summary both appear.

Chapter 21. LLM Integration and Response Generation

Retrieval selects evidence. Generation writes sentences. The client is chosen by llm.provider. settings.yaml sets provider mock and model mentor-provided-model. If the environment contains LLM_ENDPOINT and does not contain LLM_PROVIDER, load_config switches the provider to custom_api. MockLLMClient.generate_completion returns the fixed string "Mock LLM text completion. The input context was successfully received." The five stored answers are not that string. responses.md records that they were produced by Llama 3.2 through Ollama. The endpoint address is an environment setting and is not copied here.

CustomAPILLMClient posts a JSON body with model, messages, and temperature to the configured endpoint, with a 180-second timeout. The system message is RAG_SYSTEM_PROMPT. The user message on the ask path is the heading "Retrieved Unified Context:", the formatted hits, the heading "User Question:", the query, and the sentence "Answer only from the context and cite its provenance." Temperature is passed as 0.0 from answer_question. There is no max_tokens field in the payload. The response parser accepts choices[0].message.content, or a completion field, or a text field. Errors become RuntimeError. API keys, when set, are SecretStr values and are not printed by this report.

The system prompt tells the model to use only the retrieved context, to avoid outside knowledge, to preserve technical terms, and, when the context is insufficient, to answer with exactly "The information is not available in the provided context." It also asks for document names and page numbers on PDF chunks. The grounding gate may emit that same sentence without calling the model. A model that is called can still ignore the instruction; Chapter 30 treats that as a generation failure rather than a retrieval failure.

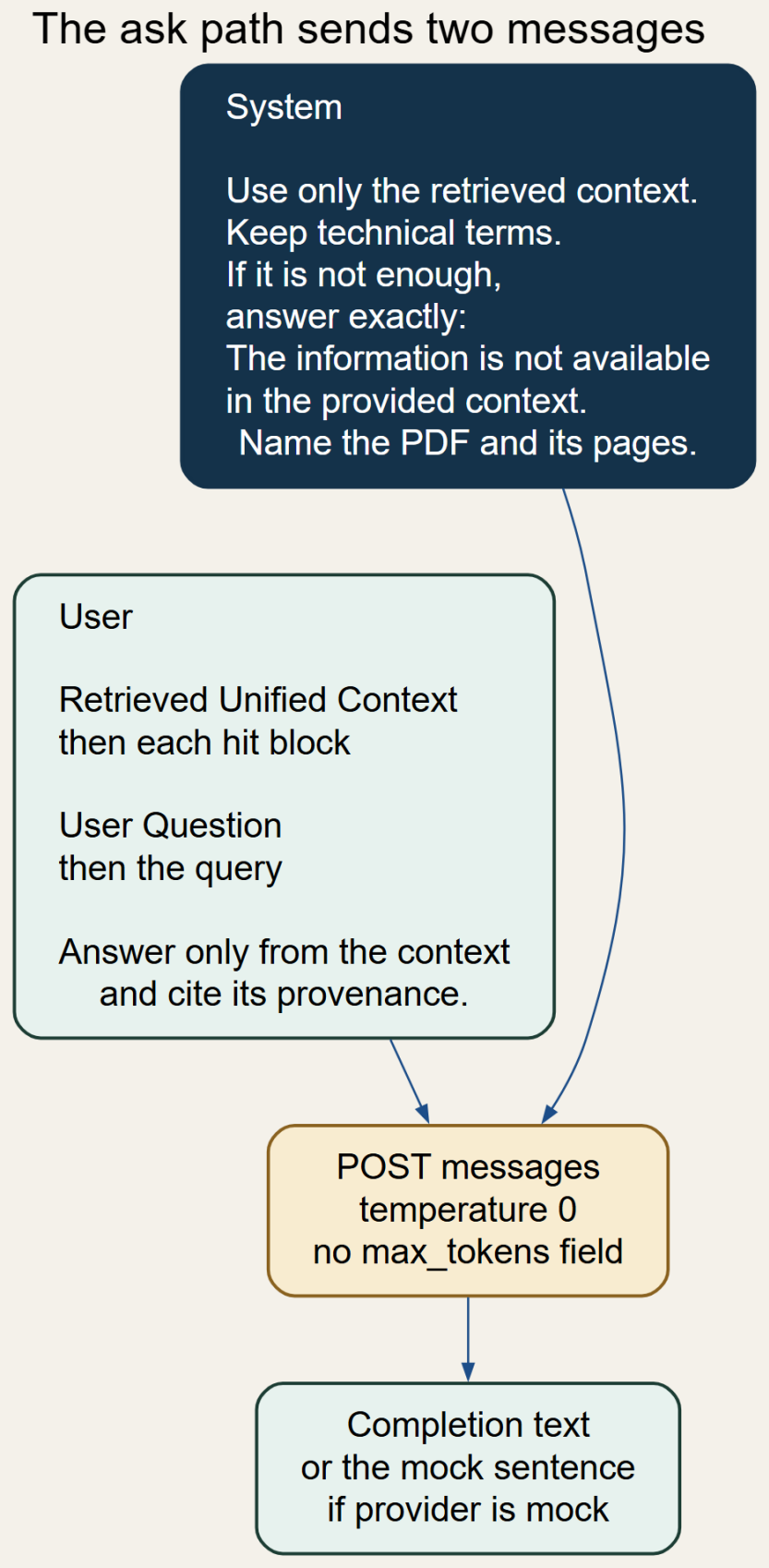


Figure 18. Prompt construction on the ask path

PDFRAGPipeline.answer_question calls RAGPipeline.answer_query. That method uses a shorter user prompt headed "Retrieved PDF Context:" and, unless a source filter or the graph flag is passed, it does not apply the score threshold. Status: client and prompts DONE. Default provider is mock. The demo run used a live model and is the generation evidence. Tests: test_pdf_providers.py and the prompt assertions in the contract tests. Limitation: no token limit is sent, and the mock client cannot validate answer quality.

Chapter 22. Citation and Source Attribution

The application citation is the sources array. Each SourceReference copies chunk_id, document_id, document_name, source_kind, content_kind, source_url, section, page_numbers, score, and retrieval_method. For the cyber-forensics PDF the third saved source is chunk doc_cc7477d8bc498b35d746bfa461d023b3357533e1845706862ad12ef53c441bc4_chunk_002, document 82f38954477f2c53.pdf, pages [1], hybrid score 0.6678465341224721. That page list was produced by the chunker and stored on the chunk. It is an application citation.

The answer text in q5.json also contains a provenance paragraph. That paragraph names a different chunk, doc_e7c001eabdb3bbdb_chunk_019, and it says the provenance does not explicitly mention cyber forensics and then draws an inference. That prose was written by the model. It is not a field generated by build_source_references. When the two disagree, the sources array is the one the application attached. Multiple sources are simply the list. Missing evidence is the abstain path: the sentence is fixed and abstained is true. None of the five saved answers abstained.

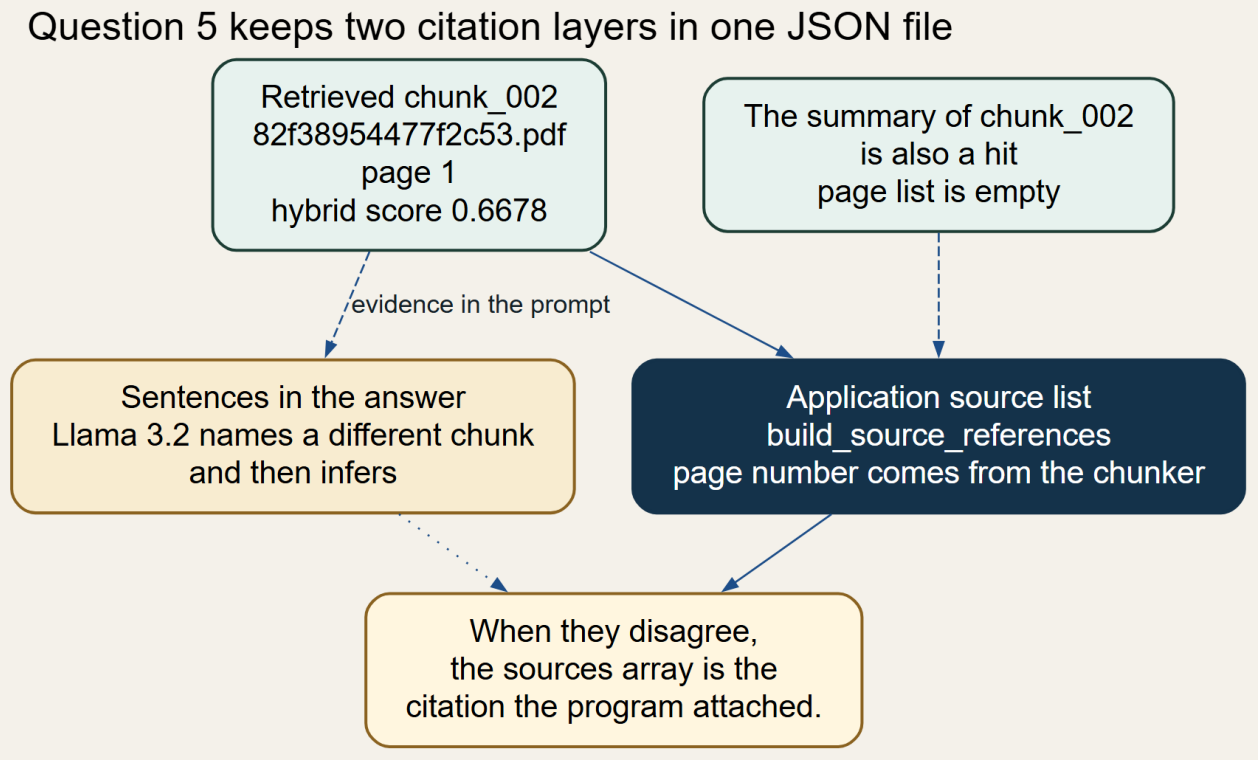


Figure 19. Two citation layers: application sources and model prose

Status: application citations DONE; agreement between model prose and those citations PARTIAL, as q5.json demonstrates. Limitation: summary hits have empty page lists, and the model is not forced to copy the sources array verbatim.

Chapter 23. End-to-End Query Execution

The order below is the order in answer_question, which is the function used by run.py ask, by POST /ask, and by docs/demo-results/_ask.py.

- 1. The caller passes the query string.
- 2. Blank text raises ValueError.
- 3. load_config has already merged settings.yaml and the environment.
- 4. get_retriever builds a HybridRetriever when mode is hybrid.

- 5. Hybrid search embeds the query inside VectorRetriever, runs BM25, runs kNN, fuses, and returns five hits.
- 6. grounding_failure inspects those corpus hits.
- 7. On failure, the function returns the abstain sentence and the corpus sources.
- 8. On success, optional knowledge-graph hits are appended.
- 9. build_rag_context formats the list.
- 10. The system prompt and the user prompt are sent to the client at temperature 0.
- 11. The stripped completion becomes RAGAnswer.answer.
- 12. build_source_references fills RAGAnswer.sources.
- 13. The object is serialised to JSON by the CLI and by the demo script.

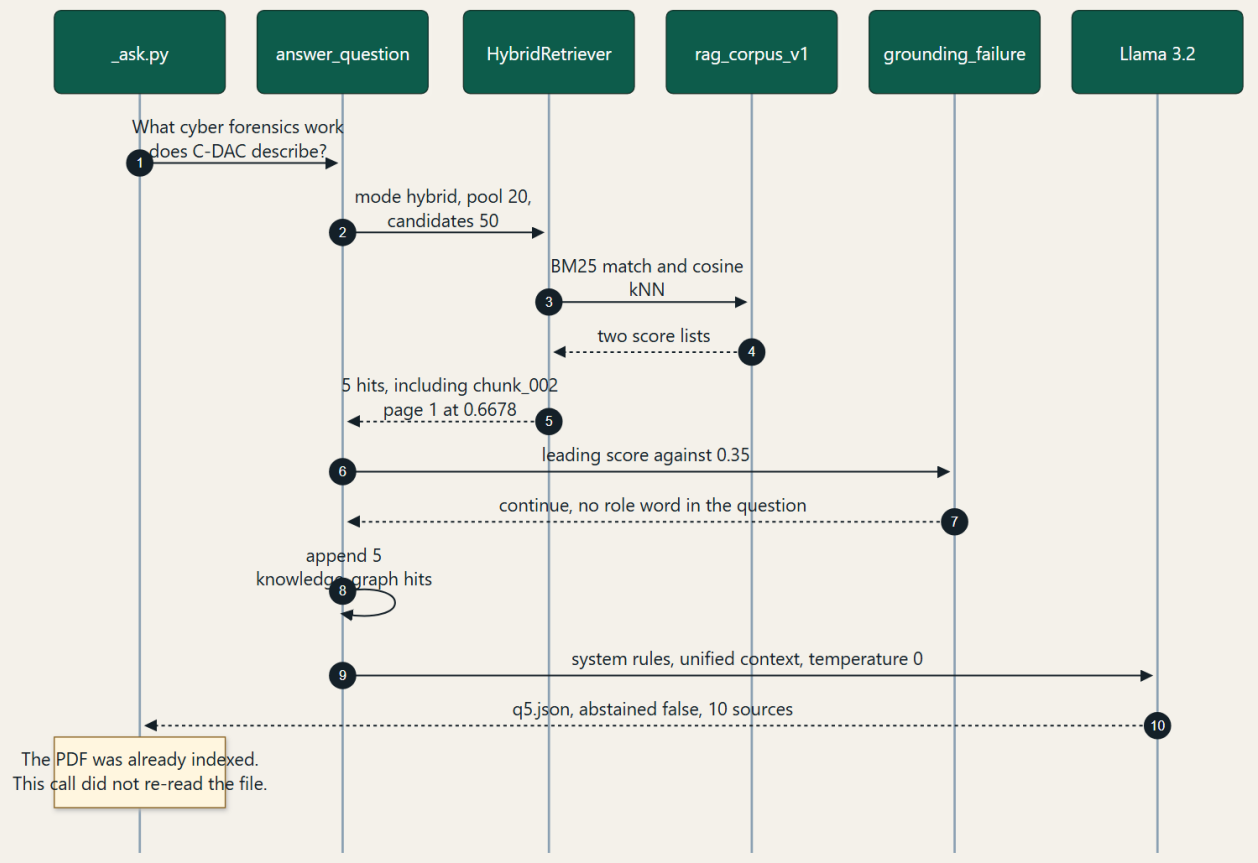


Figure 20. Recorded ask execution order

The embedding of the query happens inside step 5, not as a separate step before BM25. BM25 does not need a vector. The worked record is Chapter 38. Status: this order matches the source of answer_question. The programmatic RAGPipeline.answer_query order differs at the grounding step, as Chapter 21 states.

23.1 Passage retrieved for the worked PDF

The following text is the start of chunk 002 from the stored artifact. It is the passage whose identifier appears in q5.json.

```
C-DAC Cyber Forensics Tools

Issued by

CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING
(The Premier R&D organization of the
Ministry of Electronics and Information Technology (MeitY), Govt. of India)
Thiruvananthapuram, Kerala, 695033
Phone:0471 2727508 Fax:0471 2723456 Email: tpc@cdac.in
Website:www.cdac.in
```

```
Dated:15
th October 2026
```

Part D — Algorithms, Configuration, and Execution

Chapter 24. Algorithms and Pseudocode

Algorithms written in this repository are separated from algorithms executed inside pypdf, Sentence Transformers, Lucene, or Elasticsearch. Time and space notes are given only where the project code's loops make them direct.

Table 10. Algorithms and the module that implements each one

Algorithm	Module	Owner
1. PDF validation	pdf_processor.py	Project
2. Page-wise extraction	PDFProcessor.process_pdf	Project loop, pypdf parser
3. OCR decision	check_requires_ocr, OCRPageContext	Project decision; Tesseract if present
4. Text preprocessing	clean_extracted_text, remove_repeated_boilerplate	Project
5. Chunk splitting	Chunker.chunk_structurally	Project
6. Overlap	split_text_with_overlap and the carry-over tail	Project
7. Duplicate detection	_validate_chunk	Project
8. Chunk merging	_merge_small_chunks	Project
9. Embedding generation	LocalEmbeddingProvider	Library model; project checks dimension
10. Cosine similarity	Elasticsearch dense_vector	Engine
11. BM25 retrieval	BM25Retriever	Engine score; project query
12. HNSW search	VectorRetriever knn clause	Engine
13. Min-max normalisation	min_max_normalize	Project
14. Hybrid fusion	compute_weighted_min_max_scores	Project
15. Top-k ranking	fuse_hybrid_results	Project
16. Context construction	build_rag_context	Project
17. Prompt assembly	answer_question	Project
18. LLM generation	CustomAPILLMClient or MockLLMClient	External model, or a stub
19. Citation mapping	build_source_references	Project

1. PDF validation

Objective: Confirm a path can be hashed and opened as a non-empty PDF. Inputs: Path. Outputs: document_id and page count, or an exception.

Working principle: Read bytes, SHA-256, construct PdfReader, reject zero pages. Owner: Project, module pdf_processor.py.

Cost note: O(file size) to hash; parser cost is inside pypdf.

2. Page-wise extraction

Objective: Obtain a string per page. Inputs: PdfReader page. Outputs: Cleaned page text.

Working principle: extract_text, then the cleaners in algorithm 4. Owner: Project loop, pypdf parser, module PDFProcessor.process_pdf.

Cost note: Linear in the number of pages, plus the library parser.

3. OCR decision

Objective: Recover a page whose text layer is almost empty. Inputs: Cleaned page text. Outputs: Replaced text or IMAGE_ONLY.

Working principle: If non-space length < 15 and the binary exists, render at dpi/72 and recognise. Owner: Project decision; Tesseract if present, module check_requires_ocr, OCRPageContext.

Cost note: Rendering cost dominates; skipped entirely when the binary is missing.

4. Text preprocessing

Objective: Make page text safer to chunk and search. Inputs: Raw page string. Outputs: Cleaned string.

Working principle: Filter controls, join hyphenation, collapse whitespace, drop matched header lines after page 1. Owner: Project, module clean_extracted_text, remove_repeated_boilerplate.

Cost note: Linear in the page length.

5. Chunk splitting

Objective: Pack blocks into section-scoped groups near the size limit. Inputs: Blocks. Outputs: Chunk list.

Working principle: See the pseudocode in Chapter 12. Owner: Project, module Chunker.chunk_structurally.

Cost note: Linear in the number of blocks, with string copies at emit.

6. Overlap

Objective: Repeat a tail so a boundary cut retains local context. Inputs: Text, max 1200, overlap 100. Outputs: Slices.

Working principle: Choose a boundary, emit, start the next slice overlap characters earlier. Owner: Project, module split_text_with_overlap and the carry-over tail.

Cost note: Linear in text length. Overlap increases stored characters.

7. Duplicate detection

Objective: Drop repeated chunk text. Inputs: Candidate chunk and the recent deque. Outputs: Accept or reject.

Working principle: SHA-256 equality, then SequenceMatcher ratio against the last five chunks. Owner: Project, module _validate_chunk.

Cost note: The ratio is quadratic in the two chunk lengths and is limited to five comparisons.

8. Chunk merging

Objective: Avoid leaving a very short same-section chunk. Inputs: Accepted chunks. Outputs: Renumbered chunks.

Working principle: Merge forward, and once backward, only within one section and under 1.3 times the maximum. Owner: Project, module _merge_small_chunks.

Cost note: Linear in the chunk count.

9. Embedding generation

Objective: Map text to 384 floats. Inputs: String or list of strings. Outputs: Vectors.

Working principle: SentenceTransformer.encode. The network itself is not reimplemented. Owner: Library model; project checks dimension, module LocalEmbeddingProvider.

Cost note: Dominated by the model. Batch size defaults to 32.

10. Cosine similarity

Objective: Rank vectors by angle. Inputs: Query vector and stored vector. Outputs: Similarity score.

Working principle: Dot product divided by the product of norms. Owner: Engine, module Elasticsearch dense_vector.

Cost note: Applied by Elasticsearch, not by a Python loop over the corpus.

11. BM25 retrieval

Objective: Rank chunks by query terms. Inputs: Query string. Outputs: Search results.

Working principle: Match query on content, optional filters and should-boosts. Owner: Engine score; project query, module BM25Retriever.

Cost note: Engine cost. The project sends one search request.

12. HNSW search

Objective: Approximate the nearest vectors. Inputs: Query vector, k, num_candidates. Outputs: k hits.

Working principle: Layered greedy walk, described in Chapter 18. Owner: Engine, module VectorRetriever knn clause.

Cost note: Approximate. Exact brute force exists only in the benchmark helper.

13. Min-max normalisation

Objective: Put a score list onto 0 to 1. Inputs: Map of chunk id to score. Outputs: Normalised map.

Working principle: (score - min) / (max - min); all ones when min equals max. Owner: Project, module min_max_normalize.

Cost note: Linear in the pool size.

14. Hybrid fusion

Objective: Combine the two normalised lists. Inputs: Two score maps and the weights. Outputs: Final score per chunk.

Working principle: 0.3 * norm keyword + 0.7 * norm semantic, missing side as 0. Owner: Project, module compute_weighted_min_max_scores.

Cost note: Linear in the union of the two pools.

15. Top-k ranking

Objective: Order and cut. Inputs: Fused rows, k. Outputs: k rows.

Working principle: Sort by score descending and chunk id ascending, then slice. Owner: Project, module fuse_hybrid_results.

Cost note: O(n log n) for the pool.

16. Context construction

Objective: Render hits as a prompt section. Inputs: Ranked hits. Outputs: One string.

Working principle: Emit a labelled block per hit and join with blank lines. No token counter. Owner: Project, module build_rag_context.

Cost note: Linear in the total characters of the hits.

17. Prompt assembly

Objective: Build the system and user messages. Inputs: Query and context. Outputs: Two strings.

Working principle: Fixed system rules plus the context template in Chapter 21. Owner: Project, module answer_question.

Cost note: Linear in the context length.

18. LLM generation

Objective: Produce the answer string. Inputs: Messages and temperature 0. Outputs: Completion text.

Working principle: HTTP chat request, or the fixed mock sentence. Owner: External model, or a stub, module CustomAPILLMClient or MockLLMClient.

Cost note: Network and model cost; not measured in the saved JSON.

19. Citation mapping

Objective: Copy hit metadata into the response. Inputs: Retrieved chunks. Outputs: SourceReference list.

Working principle: Field copy, including page_numbers when the hit has them. Owner: Project, module build_source_references.

Cost note: Linear in the hit count.

Advantages and limitations of the retrieval algorithms are in Chapters 16 to 19. The shared advantage of the project-owned steps is that they are deterministic and unit-tested. The shared limitation is that layout-aware parsing, learned fusion, and a token budget are not among them.

Chapter 25. Configuration Reference

load_config reads config/settings.yaml, flattens its sections, and overrides any field whose upper-case name is present in the environment. .env is loaded from the repository root. This report does not print secret values. get_masked_dict replaces a password or API key with a mask.

Table 11. Configuration reference for the values that control the PDF path

Parameter	Configured value	Purpose	Effect of a change	Consumer
embedding.provider	local	Choose the encoder	mock disables meaning; custom_api calls EMBEDDING_ENDPOINT	get_embedding_provider

embedding.model	sentence-transformers/all-MiniLM-L6-v2	Model name	Must still emit 384 floats or the provider raises	LocalEmbeddingProvider
embedding.dimension	384	Expected vector length	Must match the model and the index	embedder, indexer, vector search
pdf_chunking.max_chunk_size	1200	Target chunk length	Larger chunks carry more context and blur the embedding	Chunker.max_size
pdf_chunking.chunk_overlap	100	Repeated tail	Zero disables the carry-over	Chunker.overlap
pdf_chunking.min_chunk_size	300	Merge threshold	Higher values merge more aggressively	Chunker._merge_small_chunks
duplicate_similarity_threshold	0.92 (code default)	Near-duplicate cut	Lower values drop more chunks	_validate_chunk
elasticsearch.url	http://localhost:9200	Cluster	Point at another cluster	ElasticsearchIndexer
elasticsearch.index	rag_corpus_v1	Index name	A new name starts empty	Indexer and retrievers
retrieval.mode	hybrid	Default retriever	bm25 or vector skips fusion	get_retriever
retrieval.top_k	5	Final hit count	More hits enlarge the prompt	HybridRetriever
retrieval.num_candidates	50	HNSW exploration	Higher can improve recall and latency	VectorRetriever
retrieval.hybrid_candidate_pool	20	Pre-fusion width	Changes min-max inputs	HybridRetriever.search
retrieval.keyword_weight	0.3	BM25 share	Must keep the sum at 1	fuse_hybrid_results
retrieval.semantic_weight	0.7	Vector share	Must keep the sum at 1	fuse_hybrid_results
retrieval.fusion_strategy	min_max	Fusion branch	rrf activates reciprocal rank fusion	fuse_hybrid_results
retrieval.rrf_k	60	RRF constant	No effect while strategy is min_max	compute_rrf_scores
llm.provider	mock	Default generator	custom_api uses LLM_ENDPOINT	get_llm_client
llm.model	mentor-provided-model	Name sent to a custom endpoint	The demo record names Llama 3.2 separately	CustomAPILLMClient
grounding.min_score	0.35	Abstain floor on the ask path	Higher abstains more often	grounding_failure
ocr.enabled	true	Allow OCR	false never calls	OCRProcessor

			Tesseract	
ocr.dpi	300	Render scale	Higher is slower and sometimes clearer	ocr_page_text
ocr.language	eng	Tesseract language	Requires the matching traineddata	image_to_string
image_extraction.min_dimension	50	Skip tiny images	Higher skips more images	extract_page_images
kb.source	all	Which artifacts are indexed	uploaded_pdf_only would hide the current corpus	load_index_records

Environment names are the field names in upper case, for example EMBEDDING_PROVIDER, ELASTICSEARCH_URL, RETRIEVAL_MODE, and LLM_PROVIDER. LLM_API_KEY and ELASTICSEARCH_PASSWORD are optional SecretStr fields. They were not read for this report.

Chapter 26. How to Run the Project

The commands below are the ones in README.md and run.py. They were confirmed against the argument parser. Elasticsearch was reachable at the configured URL during report preparation. A full re-ingest was not launched for this document; the corpus artifacts and the index were already present.

Table 12. Commands required to install, index, ask, and test

Command	What it does	Expected signal
python -m venv .venv	Create the virtual environment	A .venv directory
.\\.venv\\Scripts\\Activate.ps1	Activate it on Windows PowerShell	The prompt shows the environment
python -m pip install -r requirements.txt	Install pinned dependencies	pytest and the PDF libraries import
Copy-Item .env.example .env	Create the local environment file	A .env file; fill secrets locally and do not commit them
docker compose up -d elasticsearch	Start Elasticsearch 8.12.2	GET / returns version 8.12.2
python run.py status	Ping the cluster	JSON with elasticsearch_available
python run.py ingest --pdf <file>	Copy a PDF into data/raw and write an artifact	JSON with document_id and chunk count
python run.py ingest-all	Process every PDF in data/raw	JSON with processed count
python run.py process-pdfs	Process PDFs listed in the download manifest	JSON with source_kind crawled_pdf
python run.py index	Embed selected records and bulk-index them	successfully_indexed in the JSON
python run.py query "..." --mode hybrid --sources pdf	Retrieve without calling the LLM	grounded flag and sources
python run.py ask "..." --sources pdf	Grounded answer; graph off unless --include-kg	RAGAnswer JSON
python run.py ui --port 8765	Serve the inspector on 127.0.0.1	Listening message
python -m pytest tests -q	Run the automated suite	The recorded re-run is in Chapter 27

docker compose --profile ui up -d ui	Optional container UI published only on loopback	Port 127.0.0.1:8765

26.1 What the operator should expect

The first local embedding call downloads or loads all-MiniLM-L6-v2. Index creation prints nothing beyond the JSON result of the index command. A question against an empty index returns no hits and, on the ask path, the abstain sentence. The mock provider answers with its fixed sentence whenever the grounding gate allows a call. The saved demo is the record of a run that used a live Llama 3.2 endpoint instead.

26.2 Troubleshooting

- elasticsearch_available false: the cluster is down or ELASTICSEARCH_URL points elsewhere. status still exits 0.
- Existing index embedding dimension is incompatible: the index was built with a different dimension. Create a new index name rather than silently reusing it.
- Tesseract OCR binary not found: the Python package is installed and the executable is not on PATH. Sparse pages stay IMAGE_ONLY. The Docker image installs the binary.
- WinError 32 on a temporary PDF during tests: PyMuPDF still has the file open at fixture teardown. See Chapter 27.
- Hybrid weights must equal 1: keyword_weight and semantic_weight were changed to a pair that does not sum to 1.
- LLM_ENDPOINT is not configured: provider is custom_api and the endpoint is empty.

Python version used for the inspection and the test re-run: 3.12.10.

Chapter 27. Testing and Validation

The suite was re-run with the project virtual environment: python -m pytest tests -q --tb=no. Result: 281 passed, 2 skipped, 7 errors, 74.85 seconds. The seven errors are teardown PermissionError WinError 32 on temporary PDF files in test_pdf_extraction.py and test_port_pdf_pdf_processor.py. A focused rerun showed the test bodies as passed and the failure at path.unlink. The log line in that rerun also states that the Tesseract binary is not on PATH, and that PyMuPDF could not open one temporary file as a PDF. docs/CURRENT_STATE.md records an earlier result of 280 passed and 3 skipped. This chapter uses the re-run, because it was executed while preparing the report.

Table 13. Test areas and what a pass does not prove

Area	Test modules	What they demonstrate	Live dependency
PDF extraction	test_pdf_extraction.py, test_port_pdf_pdf_processor.py	Multi-page text, empty pages, per-page failure, JSON save	Bodies passed; teardown hit a Windows file lock
OCR	test_pdf_ocr.py, test_port_pdf_ocr_processor.py	Graceful behaviour when OCR cannot run	Tesseract binary absent in this run
References	test_pdf_references.py, test_port_pdf_reference_extractor.py	Figure, table, and equation parsing	No cluster
Chunking	test_pdf_chunker.py, test_port_pdf_chunker.py	Split, overlap, merge, duplicates	No cluster
Embeddings	test_port_pdf_embedder.py, test_pdf_providers.py	Mock vectors, dimension checks, provider selection	Local model is not required for the mock tests

Indexing	test_port_pdf_es_indexer.py	Mapping and bulk actions against a stand-in client	Does not prove the live cluster
BM25, vector, hybrid	test_port_pdf_retrieval_bm25.py, vector, hybrid	Query shape, fusion arithmetic, weight checks	Stand-in client
Prompts and abstention	test_unified_core.py, test_integration_contract.py	Prompt text, source filters, abstain without a model	No live LLM
End-to-end pipeline object	test_port_pdf_rag_pipeline.py	Ingest and answer with injected fakes	Not the live index

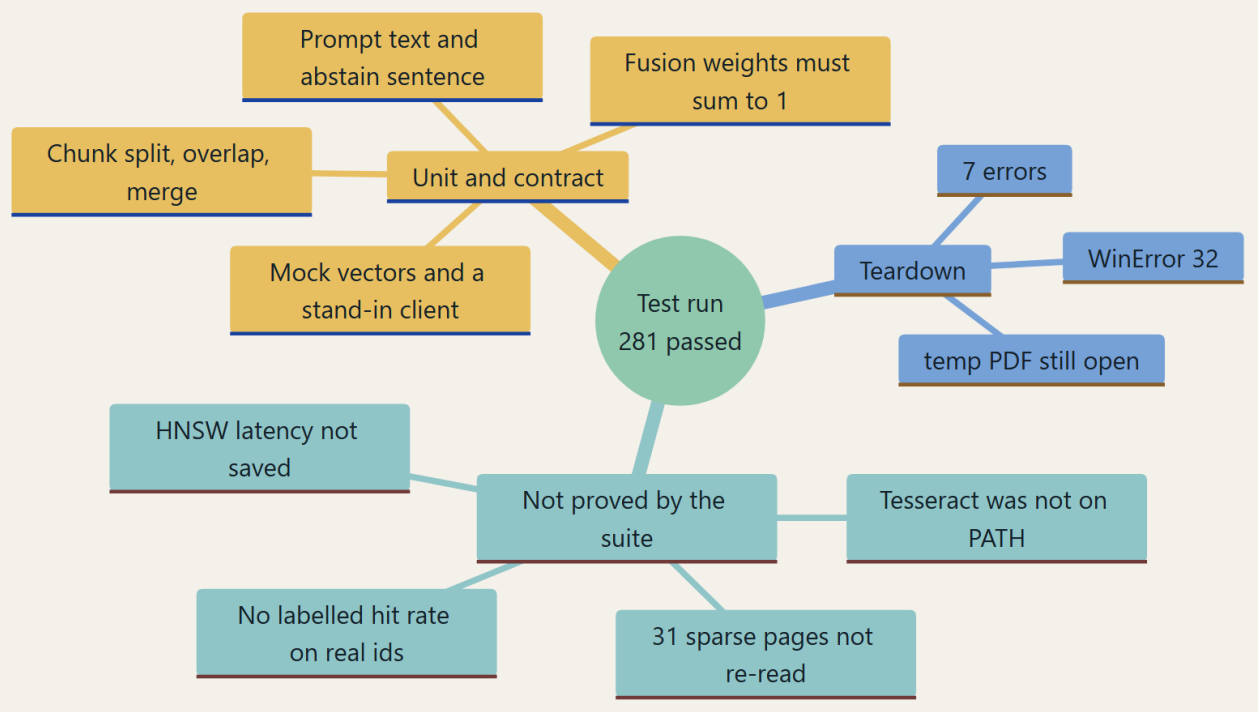


Figure 21. What the automated tests actually cover

Unit tests and contract tests are the passes above. They are not a live integration certificate. What still needs live validation is a Tesseract run on the 31 sparse pages, an HNSW benchmark with its output saved, and a labelled retrieval set whose expected document identifiers exist in rag_corpus_v1. The live facts already collected are the index count, the mapping, the artifact statistics, and the five saved answers.

Part E — Results, Evaluation, and Reliability

Chapter 28. Experimental Results

The figures in this chapter come from three places: the 60 PDF artifacts under data/extracted, the Elasticsearch index, and the JSON files in docs/demo-results. No latency field is stored in those JSON files, so query time and end-to-end time are not stated.

Table 14. Stored corpus, live index, and saved demo counts

Measurement	Value	Source
PDF artifacts with source_kind crawled_pdf	60	data/extracted/pdf_*.json
Uploaded PDF artifacts in that scan	0	source_type count
Total pages	1,479	extraction.total_pages summed
Pages counted successfully extracted	1,448	successfully_extracted_pages summed
Pages with failed status	0	failed_pages summed
Pages flagged as requiring OCR	31	pages_requiring_ocr summed
Document extraction status	SUCCESS on all 60	Because failed_pages is 0, including IMAGE_ONLY pages
PDF chunks	2,880	artifact chunk lists
PDF chunk summaries	2,880	artifact chunk_summaries
PDF composite summaries	316	artifact composite_summaries
Text-chunk length min / median / mean / max	49 / 1,073 / 933.6 / 1,481 characters	content lengths of content_kind chunk
Sum of stored extraction durations	44.38 seconds	processing_duration_seconds on the 60 extraction objects
Mean extraction duration per PDF	0.74 seconds	Same field; maximum 3.165 seconds
Sum of stored chunking durations	15.73 seconds	chunking_statistics.processing_duration_seconds
Embedding dimension	384	Live mapping and settings
Index documents	12,054	count API on rag_corpus_v1
Of which crawled_pdf	6,076	terms aggregation on source_kind
Of which html	5,978	same aggregation
content_kind chunk / summary / composite	5,616 / 5,616 / 822	Includes HTML and PDF
Saved demo questions	5	docs/demo-results/index.json
Abstained	0 of 5	each qN.json
Chunks returned per demo question	10	5 hybrid hits plus 5 knowledge-graph hits

The 6,076 crawled-PDF index documents equal 2,880 chunks + 2,880 summaries + 316 composites. That identity is the check that the artifact scan and the index agree. Extraction durations are the values written

when the artifacts were produced. They are consistent with text extraction without a long OCR pass, and they are not a new stopwatch run.

Table 15. Five answers stored in docs/demo-results

#	Question	Abstained	Hits	Mode	Source mix
1	What is C-DAC's Aadhaar Data Vault service?	no	10	hybrid	html 10; hybrid 5, knowledge_graph 5
2	Where is the C-DAC Pune centre located?	no	10	hybrid	html 9, crawled_pdf 1; hybrid 5, knowledge_graph 5
3	What has C-DAC done in High Performance Computing?	no	10	hybrid	html 10; hybrid 5, knowledge_graph 5
4	What training does the C-DAC ACTS programme provide?	no	10	hybrid	html 9, crawled_pdf 1; hybrid 5, knowledge_graph 5
5	What cyber forensics work does C-DAC describe?	no	10	hybrid	html 8, crawled_pdf 2; hybrid 5, knowledge_graph 5

Each saved answer used hybrid retrieval and included the knowledge graph, as recorded by include_kg true and by five knowledge_graph rows beside five hybrid rows. PDF hits appear in questions 2, 4, and 5. Questions 1 and 3 were answered from HTML and graph rows only. The generator named in responses.md is Llama 3.2 through Ollama.

28.1 Saved answer pages

The pages below are the screenshots stored in docs/demo-results/screenshots. Each one shows the question, the Llama 3.2 completion, the abstain flag, the hit count, and the first retrieved sources with their scores. The source cards are the application citation layer from Chapter 22.

Question 1, Aadhaar Data Vault. Ten hits, none abstained. The visible sources are HTML hybrid rows for the Aadhaar Data Vault service, scores about 0.885, 0.885, and 0.743.

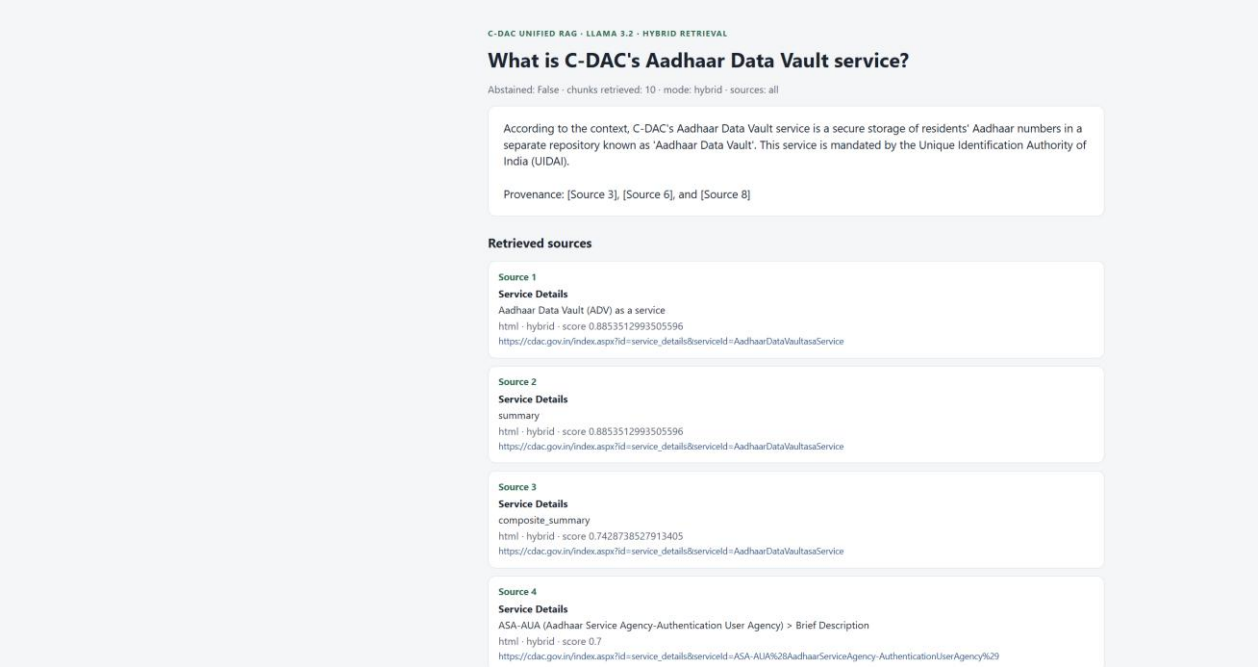


Figure 22. Saved answer page for the Aadhaar Data Vault question

Question 3, high-performance computing. The completion names a Parametric Simulation and Computing facility and then quotes a chunk id from the In the News page. The first visible source is an HTML product row for a reconfigurable computing card, score 0.7.

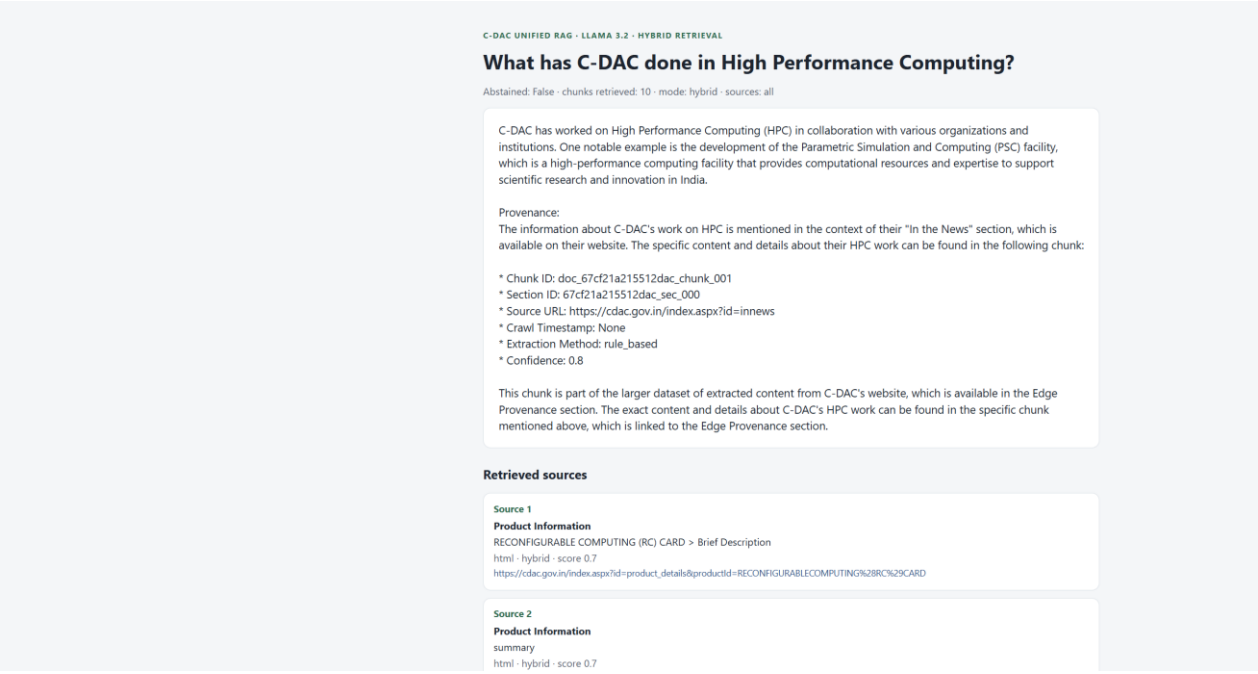


Figure 23. Saved answer page for the high-performance computing question

Question 4, ACTS training. The completion answers from a knowledge-graph row and names an In the News URL. A crawled PDF is also in the ten hits, as recorded in the JSON, further down the source list than this screenshot shows.

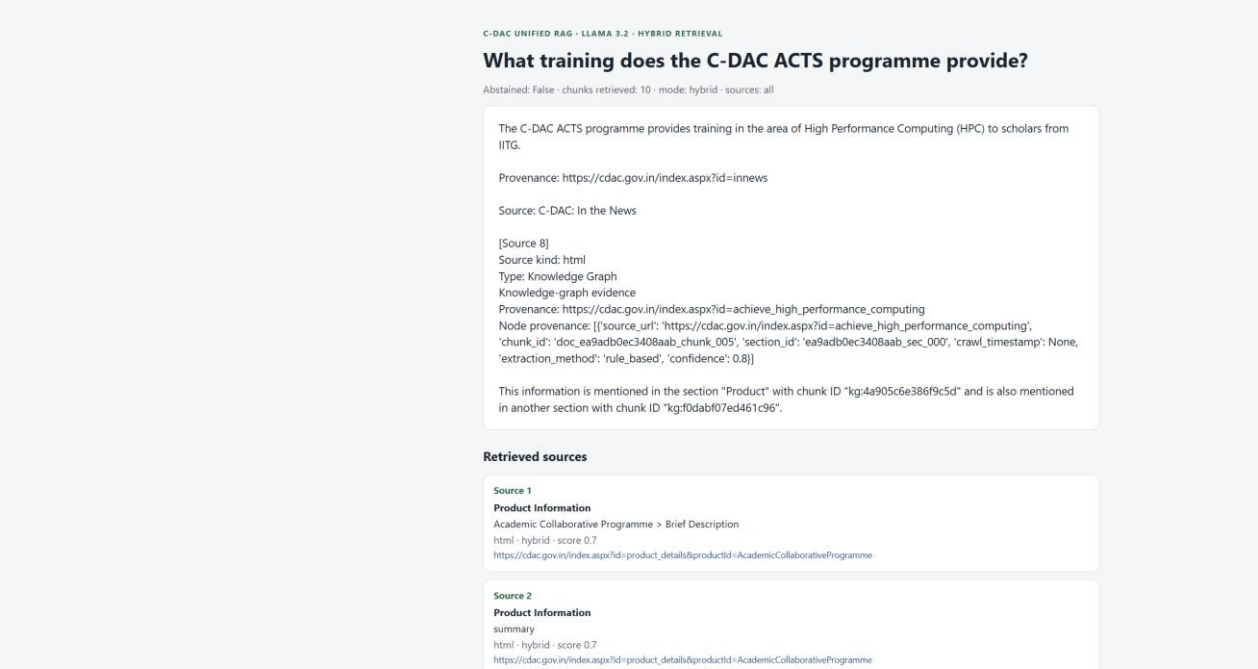


Figure 24. Saved answer page for the ACTS training question

Question 5, cyber forensics. This is the walk-through in Chapter 38. The completion infers beyond one provenance paragraph. The JSON source list, including page 1 of 82f38954477f2c53.pdf, is the application citation.

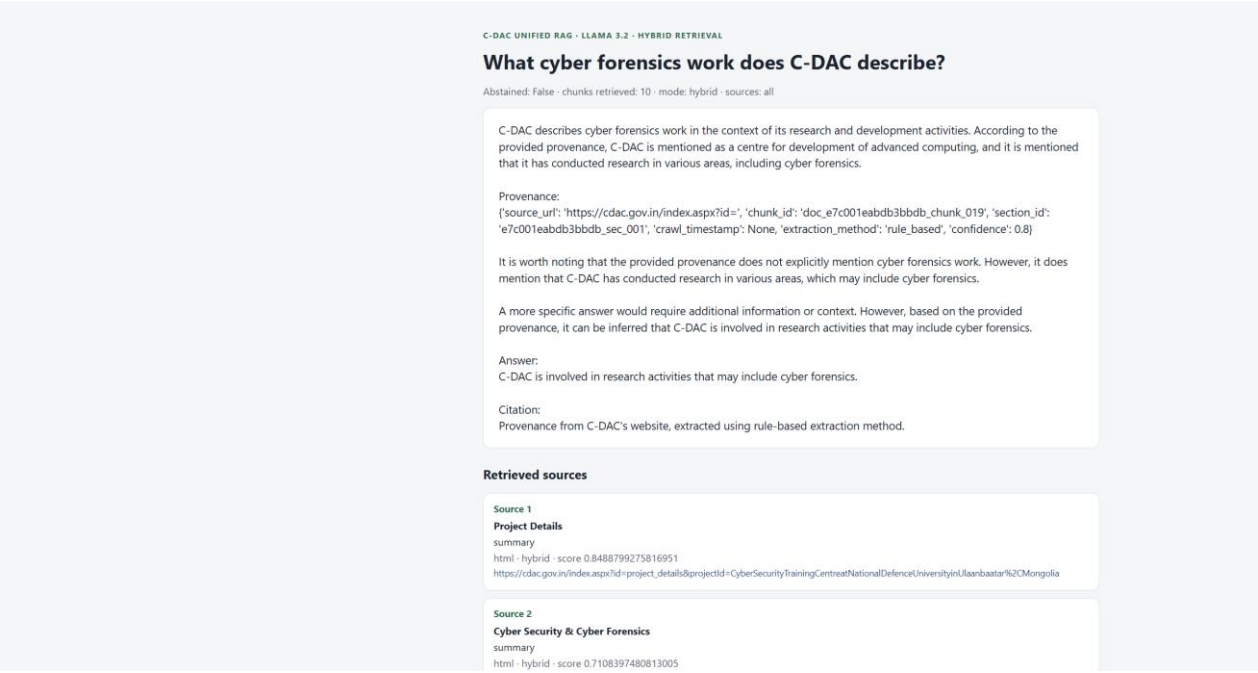


Figure 25. Saved answer page for the cyber-forensics question

The inspector dashboard screenshot shows the pipeline JSON served by the local UI. The chunk, summary, composite, and index totals used in this chapter were recomputed from the artifacts and from Elasticsearch, and they agree with the indexed total of 12,054.

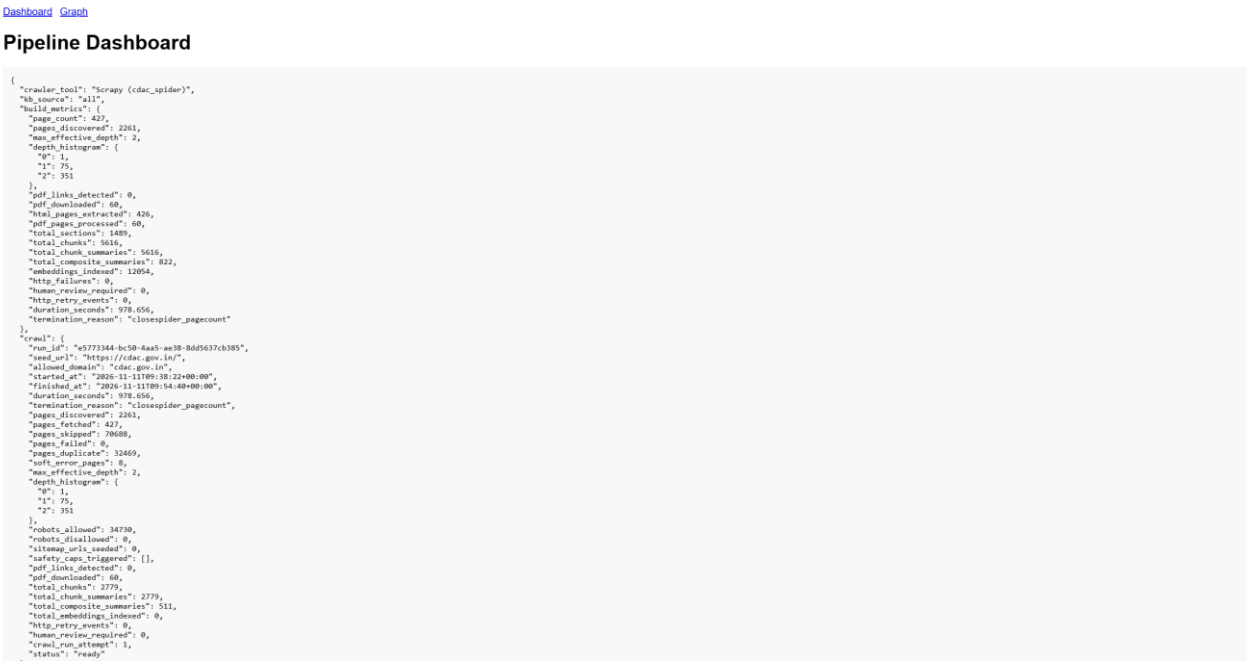


Figure 26. Saved pipeline dashboard from the local inspector

Chapter 29. Retrieval Evaluation

rag/eval/metrics.py implements hit rate, recall, and reciprocal rank. It does not implement precision or NDCG. The runner reports hit@1, hit@5, MRR, recall@5, and latency, and it can call BM25, vector search, hybrid search with the evaluation weights 0.3 and 0.7, and the knowledge graph. The fixture rag/eval/fixtures/benchmark.jsonl contains five questions whose expected document identifiers are labels such as cdac_org and cdac_hpc. Those identifiers are not the SHA-256 document identifiers in the index. No file in the repository stores an output of run_evaluation against the live index. A labelled evaluation is therefore pending.

Table 16. Retrieval metrics: definitions, code, and whether a score exists

Metric	Formula	Present in code	Scored on this corpus
Hit rate@k	1 if any of the first k retrieved document ids is in the expected set	Yes	No stored run
Recall@k	|retrieved@k ∩ expected| / |expected|	Yes	No stored run
MRR	1 / rank of the first relevant document, else 0, averaged over questions	Yes	No stored run
Precision@k	|retrieved@k ∩ expected| / k	No	Pending
NDCG@k	Discounted gain against a graded ranking, normalised by the ideal	No	Pending
Latency	Wall time around one search call	Yes, in the runner	No stored run

A one-question illustration of the formulas, not a measured score: expected set {D}, retrieved order [A, D, C], k = 3. Hit@3 is 1, recall@3 is 1, reciprocal rank is 1/2, precision@3 is 1/3. If the grades were 1 for D and 0 otherwise, NDCG would discount D because it is in position 2. That arithmetic is the definition only.

Chapter 30. Response Quality and Hallucination Control

Groundedness, in the implemented sense, is the gate in grounding_failure plus the prompt rule that forbids outside knowledge. The gate has three triggers: no corpus hits; leading score below 0.35; a question that names Executive Director, Director, Scientist, or Engineer when that same role string is absent from the hit text. The five saved answers cleared the gate. Context relevance and answer relevance are not scored by a metric in the repository.

Citation correctness has to be judged against the sources array. In q5.json the array contains a real cyber-forensics page and a PDF about reselling cyber-forensics tools. The model prose then says the cited provenance does not explicitly mention the work and still offers an inference. That is a generation-quality gap on top of a retrieval hit. It is recorded, not repaired, by this report. Abstention behaviour is implemented and was not exercised by these five questions. A retrieval failure is an abstain reason of no_retrieval_hits or score_below_threshold. A generation failure is a completion that goes beyond the hits after the gate has passed.

Safeguards in the implementation: the fixed abstain sentence, temperature 0, the source list, and the prompt rules. Safeguards that are recommendations: checking that every sentence in the answer is supported by a listed chunk, refusing inferences, and dropping summary duplicates before the prompt is built. Those checks are not in the code.

Chapter 31. Performance and Scalability

Measured from stored artifact fields: extraction of 60 PDFs took 44.38 seconds in sum, and chunking took 15.73 seconds in sum. Those sums are not a concurrent benchmark. Embedding time, bulk-index time, query latency, and LLM latency are not stored on the demo JSON and are not invented here. The index holds 12,054 vectors of 384 floats. At 4 bytes per float that is about 18.5 MB of raw vector payload before Elasticsearch overhead and HNSW links, which is a size calculation rather than a disk measurement.

Batch embedding uses batch size 32. The hybrid pool is 20 plus a vector candidate list of 50, which is small relative to 12,054 documents and is the reason an approximate index is used. Memory for the local encoder is the Sentence Transformers process working set; it was not sampled. Elasticsearch in docker-compose.yml is given a 512 MB JVM heap. Scaling beyond one node is not configured. LLM latency depends on the endpoint and is outside the retrieval timings.

The HNSW trade-off is configured only through num_candidates. No recall-versus-latency table has been produced. Status of performance numbers: the extraction and chunking sums are DONE as artifact fields; query and model latency remain NEEDS LIVE VALIDATION.

Part F — Implementation Status and Future Scope

Chapter 32. Implementation Status Matrix

Table 17. Implementation status of each major subsystem

Subsystem	Status	Evidence
PDF validation and byte identity	DONE	process_pdf exceptions; SHA-256 document ids in artifacts
pypdf page extraction	DONE	1,479 pages stored; extraction tests passed their bodies
Cleaning and header removal	DONE	clean_extracted_text and remove_repeated_boilerplate
OCR decision and fallback	PARTIAL	31 pages flagged; Tesseract absent on the test host
Image extraction to chunks	PARTIAL	Code present; image chunk sum is 0
Structural chunking, overlap, merge, near-duplicates	DONE	Chunker; cyber PDF stats show 3 merges and 0 LLM calls
LLM-assisted chunking	PENDING	Method exists; configuration leaves it off
Page provenance on text chunks	DONE	Demo PDF hit pages [1]
Page provenance on summaries	PARTIAL	Demo summary hit pages []
Local 384-dimensional embeddings	DONE	settings and live mapping
Mock and custom embedding providers	DONE as code	Not the active provider
Elasticsearch mapping and bulk index	DONE	12,054 documents, version 8.12.2
BM25 query path	DONE	BM25Retriever; hybrid demo used it inside fusion
Cosine kNN and HNSW index flag	DONE	mapping index true, similarity cosine
HNSW m, ef_construction, and a saved recall number	NEEDS LIVE VALIDATION	Not set in the mapping; benchmark has no stored output
Min-max hybrid 0.3/0.7	DONE	settings and q1–q5 retrieval_method hybrid
Reciprocal rank fusion	DONE as code, inactive	rrf_k 60; strategy min_max
Grounding gate on ask	DONE	grounding.py; demo did not abstain
Grounding gate on RAGPipeline.answer_query	PARTIAL	Score threshold is not applied on that default path
Mock LLM	DONE	Default in settings.yaml
Live generation	DONE for the five stored questions	q1.json–q5.json and responses.md
Application source list	DONE	sources arrays
Agreement of model prose with those sources	PARTIAL	q5.json inference paragraph
Token budget	PENDING	build_rag_context concatenates all hits
Labelled retrieval metrics on this index	PENDING	Fixture ids do not match the corpus

Automated tests	PARTIAL	281 passed, 2 skipped, 7 Windows teardown errors
Docker packaging	DONE as files	This report did not rebuild the image
Local UI	DONE as code	A dashboard screenshot is stored; the UI was not restarted for this report

Chapter 33. Limitations

33.1 Observed

- Thirty-one pages are still marked as requiring OCR, and the test host had no Tesseract binary.
- No image OCR chunk was produced for the 60 PDFs.
- pypdf text order is not a layout reader. Multi-column pages can merge in the wrong order.
- The heading detector and the running-head remover are regular expressions. Both can miss or over-remove.
- Summary and composite documents do not store page numbers, yet they are retrieved alongside chunks.
- Min-max weights are fixed. One outlier in a pool of 20 rescales every other score.
- The default LLM is a mock. Answer quality depends on whichever endpoint is configured.
- The model can infer beyond the hits after the gate has passed, as q5.json shows.
- Temporary PDF teardown can raise WinError 32 on Windows when PyMuPDF still holds the file.
- The PyMuPDF import uses the deprecated fitz name on version 2.1.

33.2 Structural, and not yet measured

- A 384-dimensional MiniLM vector will miss some technical paraphrases and some exact identifiers that BM25 would catch. The hybrid mix reduces that risk and does not remove it.
- Elasticsearch is a hard dependency for any live question. The status command reports the outage and does not itself answer.
- The index currently mixes HTML and PDF because kb.source is all. A PDF-only question must pass --sources pdf or uploaded_pdf.
- There is no measured limit for a PDF of many hundreds of pages beyond the linear loops in extraction and chunking.

The first group was seen in artifacts, tests, or saved answers. The second group follows from the design and should not be quoted as a measured failure rate.

Chapter 34. Future Scope and Proposed Enhancements

Every item in this chapter is proposed. None of it is described as already running.

Table 18. Proposed enhancements. These rows are not implemented features

Proposal	Current gap	Technique	Benefit	Complexity	Phase
Layout-aware extraction	Column order and tables are plain text	A layout model beside pypdf	Better chunks from reports and forms	High	1
Run OCR on the 31 sparse pages	Tesseract was not available on the test host	Install the binary and reprocess IMAGE_ONLY pages	Recover scanned pages	Low	1

Carry page numbers onto summaries	Summary hits cite no page	Copy the parent chunk page list in summary_to_indexable	Citations stay page-level	Low	1
Close the Windows file handle	Test teardown WinError 32	Guarantee OCRPageContext.close before unlink	A clean suite on Windows	Low	1
Sentence or semantic chunking	Character tails can split a thought	Split on syntactic units, or on embedding similarity	More stable passages	Medium	2
Cross-encoder reranking	Fusion has no second reader	A cross-encoder over the pool of 20	Better precision at 5	Medium	2
Query rewriting and multi-query	One string is embedded	Generate a few paraphrases and fuse	Better recall on short questions	Medium	2
Contextual compression and a token budget	All hits are concatenated	Trim passages to a token cap	More predictable prompts	Medium	2
Citation check	Model prose can drift	Require each claim to overlap a listed chunk	Fewer unsupported sentences	Medium	2
Gold question file	Fixture ids do not match the index	Label document ids from rag_corpus_v1 and run the existing metrics	A real hit-rate and recall	Medium	2
A stronger encoder	MiniLM is compact	A larger Sentence Transformers model, with a new index	Better semantic matches	Medium	3
Table and image understanding	Images produced no chunks	Table structure plus a captioner	Answers about figures	High	3
Conversation memory	Each ask is stateless	A bounded history of chunk ids	Follow-up questions	Medium	3
PDF-only serving default	The demo searched all sources	A documented --sources pdf preset	Clearer PDF evaluations	Low	3

Dependencies for phase 1 are a Tesseract install and a small indexer change. Phase 2 depends on a labelled file and, for the cross-encoder, an additional model. Phase 3 depends on a new index whenever the vector dimension changes. Agentic tool routing is not part of the current code; rag.services exposes ordinary functions, and the phase note in the README leaves tool orchestration unimplemented.

Chapter 35. Conclusion

The project implements a complete PDF path from file bytes to a grounded answer. A document is hashed, extracted page by page, cleaned, and packed into chunks that remember their pages. Those chunks are embedded with all-MiniLM-L6-v2 into 384 dimensions and stored in Elasticsearch beside a BM25 text field and a cosine HNSW vector. Questions are answered by a min-max hybrid of BM25 and vector search, with weights 0.3 and 0.7, a pool of 20, and a final cut of 5. The ask path can abstain, and it always returns the source list that the application actually retrieved.

The stored corpus of 60 PDFs, 1,479 pages, and 2,880 chunks is indexed. The live cluster held 12,054 documents in rag_corpus_v1, 6,076 of them from those PDFs. Five saved questions, answered by Llama 3.2 through Ollama with hybrid retrieval, are the generation record. The automated suite passed 281 tests, with seven Windows teardown errors that did not fail the assertions. What remains open is OCR on the sparse pages, page numbers on summary hits, a labelled retrieval score, and a check that the model's sentences stay inside the source list. Those gaps are specified in Chapters 32 to 34 and are the practical continuation of the work.

Part G — Appendices and Technical References

Chapter 36. Full Module Reference

The inventory lists the modules a reviewer needs for the PDF path. Classes and functions are the public ones. Crawl and graph modules exist in the tree and are summarised in one row so the boundary stays explicit.

Table 19. Module inventory for the PDF path

Module	Main types and functions	Role
rag/config.py	AppConfig, load_config, get_masked_dict	Validated settings
rag/models.py	DocumentModel, PageModel, BlockModel, ChunkModel, ChunkingStatistics	Shared records
rag/ingestion.py	process_pdf, ingest_uploaded_pdf, ingest_all_uploaded_pdfs, index_corpus	Artifact and index orchestration
rag/pdf_rag_pipeline.py	PDFRAGPipeline, ingest_pdf, answer_question, run_end_to_end	Programmatic one-call path
rag/extract/pdf_processor.py	PDFProcessor, clean_extracted_text, check_requires_ocr, detect_block_type	Text extraction
rag/extract/ocr_processor.py	OCRProcessor, OCRPageContext, ocr_dependencies_available	Optional OCR and images
rag/extract/reference_extractor.py	extract_references, extract_query_references, is_visual_query	Figure and table numbers
rag/index/pdf_chunker.py	Chunker, split_text_with_overlap	PDF chunks
rag/index/embedder.py	Mock, Local, and CustomAPI providers, get_embedding_provider	Vectors
rag/index/indexer.py	ElasticsearchIndexer, chunk_to_indexable, index_records	Mapping and bulk writes
rag/index/summary_builder.py	extractive_summary, build_chunk_summaries, build_composite_summaries	Deterministic summaries
rag/retrieve/bm25.py	BM25Retriever.search	Keyword search
rag/retrieve/vector.py	VectorRetriever.search	kNN search
rag/retrieve/hybrid.py	HybridRetriever, min_max_normalize, fuse_hybrid_results, compute_rrf_scores	Fusion
rag/retrieve/factory.py	get_retriever	Mode switch
rag/retrieve/filters.py	bm25_query, knn_query, source_domain_filter	Source and domain filters
rag/answer/pipeline.py	answer_question, RAGPipeline, build_rag_context, RAG_SYSTEM_PROMPT	Answers
rag/answer/grounding.py	grounding_failure, ABSTAIN_ANSWER	Pre-generation gate
rag/answer/llm_client.py	MockLLMClient, CustomAPILLMClient,	Generation clients

	get_llm_client	
rag/services.py	search_corpus, answer_question	CLI and UI entry
rag/ui.py	launch_ui	Local HTTP inspector and POST /ask
rag/eval/metrics.py	hit_at_k, recall_at_k, reciprocal_rank	Metric definitions
rag/eval/runner.py	run_evaluation	Comparison harness; no stored live score
rag/eval/hnsw_benchmark.py	run_hnsw_benchmark, brute_force_top_k	Synthetic latency harness
run.py	build_parser, cmd_ask, cmd_index, cmd_ingest	Operator CLI
rag/crawl and rag/graph	Spider, download store, KnowledgeGraphStore	Present in the repository; outside the detailed PDF chapters. The demo called the graph.

Chapter 37. Sample Data and Execution Outputs

The samples are taken from 82f38954477f2c53.pdf and from docs/demo-results/q5.json.

Table 20. Worked PDF record used in the demo

Item	Recorded value
File	data\pdfs\82f38954477f2c53.pdf
pdf id	82f38954477f2c53
document_id	cc7477d8bc498b35d746bfa461d023b3357533e1845706862ad12ef53c441bc4
Pages	25 extracted, 0 failed, 0 OCR-flagged
Blocks in	95
Chunks out	61; mean length 842; 3 merges; 0 LLM calls
Extraction seconds stored	0.700
Chunking seconds stored	0.304
Retrieved chunk	chunk_002, section heading about reselling software and hardware, page [1]
Hybrid score stored	0.6678465341224721
Companion summary hit	sum_..._chunk_002, page list empty, same score
Embedding	384 floats in the index; the vector itself is not printed

Opening of the retrieved chunk:

```
C-DAC Cyber Forensics Tools

Issued by

CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING
(The Premier R&D organization of the
Ministry of Electronics and Information Technology (MeitY), Govt. of India)
Thiruvananthapuram, Kerala, 695033
Phone:0471 2727508 Fax:0471 2723456 Email: tpc@cdac.in
Website:www.cdac.in
```

```
Dated:15
th October 2026
```

Opening of the stored model answer for that question:

```
C-DAC describes cyber forensics work in the context of its research and development activities.
According to the provided provenance, C-DAC is mentioned as a centre for development of advanced
computing, and it is mentioned that it has conducted research in various areas, including cyber
forensics.

Provenance:
{'source_url': 'https://cdac.gov.in/index.aspx?id=', 'chunk_id': 'doc_e7c001eabdb3bbdb_chunk_019',
'section_id': 'e7c001eabdb3bbdb_sec_001', 'crawl_timestamp': None, 'extraction_method':
'rule_based', 'confidence': 0.8}

It is worth noting that the provided provenance does not explicitly mention cyber forensics work.
However, it does mention that C-DAC has conducted research in various areas, which may include
cyber forensics.

A more specific answer would require additional information or context. However, based on the
provided provenance, it can be inferred that C-DAC is involve
```

BM25-only and vector-only lists for this question were not saved. The stored retrieval method on the corpus rows is hybrid, which means those two lists were computed and then fused. Chapter 19 gives the fusion arithmetic. The final context is the ordered sources in q5.json, five hybrid rows followed by five knowledge-graph rows, because the demo included the graph.

Chapter 38. Sample Query Walkthrough

Question, stored in q5.json: "What cyber forensics work does C-DAC describe?"

- The PDF was already in data/pdfs and had been extracted to data/extracted/pdf_82f38954477f2c53.json.
- Extraction produced 25 pages of text and no IMAGE_ONLY page for this file.
- Structural chunking produced 61 chunks. Chunk 001 is the short invitation line on page 1. Chunk 002 carries the expression-of-interest heading and page [1].
- The CLI index path embedded that chunk with the local 384-dimensional model and stored it under its chunk id. The same text was also summarised, and the summary was stored as a second document without page numbers.
- At question time the hybrid retriever embedded the question, collected a BM25 pool and a vector pool, min-max normalised them, and applied weights 0.3 and 0.7.
- Chunk 002 survived the cut with fused score 0.6678. Its summary survived with the same recorded score. HTML pages about cyber security were ranked as well.
- The leading score is above 0.35, so the gate called the model. The demo then added five graph rows.
- Llama 3.2 wrote the answer stored in q5.json. The application citation for the PDF is page 1 of 82f38954477f2c53.pdf. The model's own provenance paragraph names a different chunk and should be read as generation text.

This walkthrough explains the stored intermediates. It does not add a retrieval score that the JSON does not contain.

Chapter 39. Diagram Index

The diagrams in the body were drawn from the modules named in each caption. They show control and data direction for the implementation, including the points where Tesseract, RRF, and the LLM are optional.

- Figure 1. System layers of the PDF question-answering path
- Figure 2. End-to-end path from a PDF file to a grounded answer

- Figure 3. Data lineage retained from the file to the index
- Figure 4. How a PDF enters the indexed corpus
- Figure 5. Page-wise text extraction
- Figure 6. OCR decision for a sparse page
- Figure 7. Text cleaning before chunking
- Figure 8. Structural chunking with overlap and merge
- Figure 9. What stays attached to a PDF chunk
- Figure 10. From chunk text to a 384-dimensional vector
- Figure 11. One chunk becomes one Elasticsearch document
- Figure 12. Keyword retrieval
- Figure 13. Semantic retrieval
- Figure 14. HNSW search, conceptual, matching the Elasticsearch dense_vector path
- Figure 15. Hybrid retrieval and ranking
- Figure 16. Illustrative min-max fusion with the active weights
- Figure 17. From ranked hits to the prompt context
- Figure 18. Prompt construction on the ask path
- Figure 19. Two citation layers: application sources and model prose
- Figure 20. Recorded ask execution order
- Figure 21. What the automated tests actually cover
- Figure 22. Saved answer page for the Aadhaar Data Vault question
- Figure 23. Saved answer page for the high-performance computing question
- Figure 24. Saved answer page for the ACTS training question
- Figure 25. Saved answer page for the cyber-forensics question
- Figure 26. Saved pipeline dashboard from the local inspector

Chapter 40. Setup and Command Reference

Consolidated operator commands. Explanations are in Chapter 26.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
docker compose up -d elasticsearch
python run.py status
python run.py ingest --pdf path\to\file.pdf
python run.py ingest-all
python run.py process-pdfs
python run.py index
python run.py query "question" --mode hybrid --sources pdf --top-k 5
python run.py ask "question" --mode hybrid --sources pdf
python run.py ui --port 8765
python -m pytest tests -q
docker compose --profile ui up -d ui
```

POST /ask accepts JSON with query, and optional top_k, mode, sources, domain, and include_kg. The UI listens on 127.0.0.1. The Compose file publishes that port on the host loopback only.

Chapter 41. References

The citations name the primary papers and the official manuals for the technologies the code actually uses. They are reading references for the algorithms, not evidence that a particular metric was achieved by this project.

- Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 2020.
- Robertson, S., and Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009.
- Malkov, Y. A., and Yashunin, D. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.
- Reimers, N., and Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP, 2019.
- Sentence Transformers documentation, model all-MiniLM-L6-v2. https://www.sbert.net/ and https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Elasticsearch 8.12 documentation: dense vector field type, kNN search, and BM25 similarity. https://www.elastic.co/guide/en/elasticsearch/reference/8.12/index.html
- pypdf documentation. https://pypdf.readthedocs.io/
- PyMuPDF documentation. https://pymupdf.readthedocs.io/
- Smith, R. An Overview of the Tesseract OCR Engine. Proceedings of ICDAR, 2007. Tesseract manual: https://tesseract-ocr.github.io/
- Python 3.12 documentation. https://docs.python.org/3.12/
- Pydantic v2 documentation. https://docs.pydantic.dev/
- Järvelin, K., and Kekäläinen, J. Cumulated Gain-Based Evaluation of IR Techniques. ACM Transactions on Information Systems, 2002. Cited for NDCG, which this project has not implemented.
- Docker Compose specification, as used by docker-compose.yml in the repository. https://docs.docker.com/compose/

Project evidence used while writing this report, all inside C:\CDAC\RAG unless noted: config/settings.yaml, requirements.txt, run.py, the rag package, tests, data/extracted PDF artifacts, docs/demo-results/q1.json through q5.json, docs/demo-results/responses.md, and a read of Elasticsearch 8.12.2 at http://localhost:9200.